LIFETRACK · INTERVIEW PREPARATION

The Complete Interview Guide

ASP.NET Core 8 Web API · Microservices Architecture · Angular v20

Grounded entirely in the **LifeTrack clinical-trial management platform** — 8 ASP.NET Core microservices behind an Ocelot gateway with an Angular frontend.

Prepared for **Hari Hara Sudhan** · Cognizant ACE Team 2026.

101 ASP.NET Core Q&A | 100 Microservices Q&A | 110 Angular Q&A | 19 source files explained

Contents

> Total: **311 interview questions** grounded in the LifeTrack clinical-trial platform, plus a full source walkthrough.

Part 0 · Project Code Walkthrough

Every source file examined in this session, reproduced with a block-by-block explanation. This is the actual LifeTrack code — read it alongside the Q&A that follows.

Architecture Overview

LifeTrack is a clinical-trial management platform split into 8 ASP.NET Core microservices behind an Ocelot API gateway, with an Angular single-page-application frontend. All inter-service and client traffic is REST/JSON. Authentication is stateless JWT. The table below is the service map discovered from ocelot.json, the launchSettings and Program.cs files.

Service	Port	Database	Responsibility
Gateway.API	5046	—	Ocelot reverse proxy / single entry point
Authentication.API	5033	GovernanceDb	Login, registration, JWT issuance, users, audit logs
ProtocolSite.API	5193	ClinicalDb	Protocols, sites, site-protocol assignments
Patient.API	5095	ClinicalDb	Patients, enrollments, visits
AdverseEvent.API	5159	ClinicalDb	Adverse events, protocol deviations
DocumentCompliance.API	5232	GovernanceDb	Regulatory documents, compliance
AnalyticsReport.API	5062	ClinicalDb	KPI reports / analytics
Notification.API	5210	Notification DB	In-app notifications, DM badge counts

Ownership split: Hari owns AdverseEvent.API, DocumentCompliance.API and AnalyticsReport.API. Naveen owns Patient.API and ProtocolSite.API. Authentication.API, Gateway.API, Notification.API and the Shared.CL class library are shared infrastructure.

Authentication.API/Program.cs

PURPOSE The composition root of the Authentication service. It wires up controllers, global filters, Swagger, CORS, caching, the EF Core DbContext, JWT settings, a named HTTP client for calling Patient.API, and the DI registrations, then builds the middleware pipeline.

```
using System.Text.Json.Serialization;
using Authentication.API.Data;
using Authentication.API.Repositories;
using Authentication.API.Repositories.Interfaces;
using Authentication.API.Services;
using Authentication.API.Services.Interfaces;
using Microsoft.EntityFrameworkCore;
using Microsoft.OpenApi.Models;
using Shared.CL.Extensions;
using Shared.CL.Filters;

var builder = WebApplication.CreateBuilder(args);

// MVC + shared filters
builder.Services.AddControllers(o =>
{
    o.Filters.Add<JwtAuthFilter>();          // JWT Bearer token validation (filter-based)
    o.Filters.Add<ActivityLogFilter>();      // Logs every successful request
    o.Filters.Add<DomainExceptionFilter>();  // DomainException -> 400, NotFoundException -> 404
    o.Filters.Add<ValidateModelFilter>();    // Invalid ModelState -> 422
})
.AddJsonOptions(o => o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));

// Swagger with Bearer security
builder.Services.AddSwaggerGen(c =>
```

```
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "LifeTrack - Authentication API", Version = "v1" });
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization", Type = SecuritySchemeType.Http,
        Scheme = "bearer", BearerFormat = "JWT", In = ParameterLocation.Header
    });
    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        { new OpenApiSecurityScheme { Reference = new OpenApiReference
            { Type = ReferenceType.SecurityScheme, Id = "Bearer" } }, Array.Empty<string>() }
    });
});

// CORS — explicit allow-list (not AllowAnyHeader/Method)
builder.Services.AddCors(options =>
{
    options.AddPolicy("AngularDevClient", policy =>
        policy.WithOrigins("http://localhost:4200", "http://127.0.0.1:4200",
                           "http://localhost:62536", "http://127.0.0.1:62536")
              .WithHeaders("Authorization", "Content-Type", "Accept", "X-Requested-With")
              .WithMethods("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"));
});

builder.Services.AddMemoryCache();
builder.Services.AddHttpContextAccessor();

// Database
builder.Services.AddDbContext<AuthDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("GovernanceDb")));

// JWT settings (bound from configuration, used by JwtTokenService)
var jwtSection = builder.Configuration.GetSection("Jwt");
builder.Services.Configure<JwtSettings>(jwtSection);

// Named HTTP client for inter-service calls to Patient.API
builder.Services.AddHttpClient("PatientApi", client =>
{
    var baseUrl = builder.Configuration["ServiceUrls:PatientApi"] ?? "http://localhost:5095/";
    client.BaseAddress = new Uri(baseUrl.TrimEnd('/') + "/");
    client.Timeout = TimeSpan.FromSeconds(10);
});

// Application services (Scoped = one instance per request)
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IAuditLogRepository, AuditLogRepository>();
builder.Services.AddScoped<IPasswordHasher, BCryptPasswordHasher>();
builder.Services.AddScoped<IJwtTokenService, JwtTokenService>();
builder.Services.AddScoped<IAuditLogService, AuditLogService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IUserService, UserService>();

var app = builder.Build();

app.UseGlobalExceptionHandler();      // first — catches everything below
app.UseCors("AngularDevClient");

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(c => c.SwaggerEndpoint("/swagger/v1/swagger.json", "Authentication API v1"));
}

// Create schema + seed default users on startup
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AuthDbContext>();
    await db.Database.EnsureCreatedAsync();
    await DbSeeder.SeedAsync(db);
}
```

```
app.MapControllers();
app.Run();
```

Line-by-line / block-by-block explanation

`using ... ;`	The using directives import the service's own layers (Data/Repositories/Services) plus EF Core, OpenAPI, and crucially Shared.CL.Extensions and Shared.CL.Filters — the shared library that supplies the cross-cutting filters and middleware every service reuses.
`WebApplication.CreateBuilder(args)`	Creates the builder that sets up the dependency-injection container, configuration sources (appsettings.json, environment variables, command line) and the Kestrel web server.
`AddControllers(o => o.Filters.Add<...>())`	Registers MVC controllers and attaches FOUR global filters in order: JwtAuthFilter validates the Bearer token; ActivityLogFilter logs each successful call; DomainExceptionFilter maps business exceptions to 400/404; ValidateModelFilter returns 422 on invalid models. Because they are global, no controller needs to repeat them.
`.AddJsonOptions(... JsonStringEnumConverter ...)`	Makes enums serialize as their names ('Patient' instead of 4) so JSON payloads are self-documenting for the Angular client.
`AddSwaggerGen(...)`	Generates interactive API docs. The Bearer security definition adds an 'Authorize' button in Swagger UI so a tester can paste a JWT and call protected endpoints.
`AddCors(... 'AngularDevClient' ...)`	Allows the Angular dev origin (localhost:4200) to call the API. An EXPLICIT allow-list of origins, headers and methods is used instead of AllowAnyOrigin/Header to keep the attack surface small. OPTIONS is included so browser pre-flight requests succeed.
`AddMemoryCache / AddHttpContextAccessor`	Registers an in-process cache (for user-list caching) and the HttpContext accessor (so the DbContext can read the current user's ID from the JWT to stamp audit rows).
`AddDbContext<AuthDbContext> (UseSqlServer(GovernanceDb))`	Registers the EF Core context pointed at the GovernanceDb connection string. Authentication and DocumentCompliance both live in GovernanceDb (governance/regulatory data).
`Configure<JwtSettings>(jwtSection)`	Binds the 'Jwt' configuration section (Secret, Issuer, Audience, ExpiryMinutes) to a strongly typed options object injected into JwtTokenService.
`AddHttpClient('PatientApi', ...)`	Registers a pooled, named HttpClient with a 10-second timeout for calling Patient.API during patient self-registration. Using IHttpClientFactory avoids socket exhaustion. The timeout is the timeout pattern — a slow Patient.API can't block the auth thread forever.
`AddScoped<IInterface, Impl>()`	Every repository and service is registered Scoped — one instance per HTTP request, which is the correct lifetime for anything that touches the DbContext.
`app.UseGlobalExceptionHandler() (first)`	Registered first so it wraps the entire pipeline; any unhandled exception below it is converted to a consistent ErrorDetail JSON response.
`EnsureCreatedAsync + DbSeeder.SeedAsync`	On startup it creates the schema if missing and seeds default users/roles. Note: EnsureCreatedAsync does NOT run migrations — for production the comment recommends 'dotnet ef database update' instead.
`app.MapControllers(); app.Run();`	Maps the attribute-routed controllers and starts listening.

Authentication.API/Services/JwtTokenService.cs

PURPOSE Generates a signed JWT for a logged-in user. The token carries identity and role claims and is signed with HMAC-SHA256 using the shared secret.

```
public class JwtTokenService : IJwtTokenService
{
    private readonly JwtSettings _settings;
    public JwtTokenService(IOptions<JwtSettings> options) => _settings = options.Value;

    public (string Token, DateTime ExpiresAtUtc) Generate(User user)
    {
```

```
        var expires = DateTime.UtcNow.AddMinutes(_settings.ExpiryMinutes);
        var roleName = user.RoleNavigation?.RoleName ?? GetRoleName(user.RoleID);
        var claims = new List<Claim>
        {
            new(JwtRegisteredClaimNames.Sub,   user.UserID.ToString()),
            new(JwtRegisteredClaimNames.Email, user.Email),
            new(JwtRegisteredClaimNames.Jti,   Guid.NewGuid().ToString()),
            new(ClaimTypes.NameIdentifier,     user.UserID.ToString()),
            new(ClaimTypes.Name,               user.Name),
            new(ClaimTypes.Email,              user.Email),
            new(ClaimTypes.Role,               roleName)
        };
        var key   = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Secret));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        var token = new JwtSecurityToken(
            issuer: _settings.Issuer, audience: _settings.Audience,
            claims: claims, notBefore: DateTime.UtcNow, expires: expires,
            signingCredentials: creds);
        return (new JwtSecurityTokenHandler().WriteToken(token), expires);
    }

    private static string GetRoleName(int roleId) => roleId switch
    {
        1 => "Admin", 2 => "ClinicalTrialManager", 3 => "Investigator",
        4 => "Patient", 5 => "RegulatoryOfficer", 6 => "DataManager", _ => "Unknown"
    };
}
```

Line-by-line / block-by-block explanation

`IOptions<JwtSettings>`	The JWT secret/issuer/audience/expiry are injected via the options pattern (set up by Configure<JwtSettings> in Program.cs), keeping configuration out of the class.
`expires = UtcNow.AddMinutes(ExpiryMinutes)`	Computes the absolute expiry in UTC. The frontend later reads this from the 'exp' claim to know when the session ends.
`claims list`	The token embeds Sub + NameIdentifier (the user id), Email, Name, Jti (a unique GUID per token enabling future revocation), and Role. The Role claim is what the backend RoleAuthorizeFilter and the Angular RoleGuard read to make authorization decisions.
`SymmetricSecurityKey + HmacSha256`	The token is signed with a symmetric key using HMAC-SHA256. The same secret validates the signature on every service — that's what makes JWT validation stateless and decentralized.
`JwtSecurityTokenHandler().WriteToken(token)`	Serializes the token to the compact 'header.payload.signature' string returned to the client.
`GetRoleName switch`	A fallback that maps the numeric RoleID to a name when the Role navigation property isn't loaded — defensive coding so a token is never issued with a blank role.

Authentication.API/Services/AuthService.cs

PURPOSE The business logic for register, login and change-password. It hashes/verifies passwords with BCrypt, records audit entries, and builds the AuthResponse (token + session user).

```
public class AuthService : IAuthService
{
    private readonly IUserRepository _users;
    private readonly IPasswordHasher _hasher;
    private readonly IJwtTokenService _jwt;
    private readonly IAuditLogService _audit;

    public AuthService(IUserRepository users, IPasswordHasher hasher,
                       IJwtTokenService jwt, IAuditLogService audit)
    { _users = users; _hasher = hasher; _jwt = jwt; _audit = audit; }

    public async Task<UserResponse> RegisterAsync(RegisterRequest req)
```

```
    {
        var email = req.Email.Trim().ToLowerInvariant();
        if (await _users.EmailExistsAsync(email))
            throw new AuthException("A user with this email already exists.");

        var user = new User
        {
            Name = req.Name.Trim(), Email = email, RoleID = req.RoleID,
            Phone = req.Phone, PasswordHash = _hasher.Hash(req.Password), IsActive = true
        };
        await _users.AddAsync(user);
        var loaded = await _users.GetByIdAsync(user.UserID) ?? user;
        await _audit.RecordAsync(user.UserID, "USER_REGISTERED");
        var roleName = loaded.RoleNavigation?.RoleName ?? GetRoleName(loaded.RoleID);
        return new UserResponse { /* ...maps fields... */ };
    }

    public async Task<AuthResponse> LoginAsync(LoginRequest req)
    {
        var email = req.Email.Trim().ToLowerInvariant();
        var user = await _users.GetByEmailAsync(email);
        if (user is null || !_hasher.Verify(req.Password, user.PasswordHash))
            throw new AuthException("Invalid email or password.");
        if (!user.IsActive)
            throw new AuthException("Your login has been blocked by the admin.");
        await _audit.RecordAsync(user.UserID, "USER_LOGIN");
        return BuildAuthResponse(user);
    }

    public async Task ChangePasswordAsync(long userId, ChangePasswordRequest req)
    {
        var user = await _users.GetByIdAsync(userId)
            ?? throw new AuthException("User not found.");
        if (!_hasher.Verify(req.CurrentPassword, user.PasswordHash))
            throw new AuthException("Current password is incorrect.");
        user.PasswordHash = _hasher.Hash(req.NewPassword);
        await _users.UpdateAsync(user);
        await _audit.RecordAsync(userId, "PASSWORD_CHANGED");
    }

    private AuthResponse BuildAuthResponse(User user)
    {
        var (token, expiresAt) = _jwt.Generate(user);
        var roleName = user.RoleNavigation?.RoleName ?? GetRoleName(user.RoleID);
        return new AuthResponse
        {
            Token = token, ExpiresAtUtc = expiresAt,
            User = new SessionUser { UserID = user.UserID, Name = user.Name,
                                     Email = user.Email, Role = roleName }
        };
    }
}
```

Line-by-line / block-by-block explanation

`Constructor injection`	Four dependencies are injected by interface — the user repository, password hasher, JWT service and audit-log service. The class depends on abstractions, not concrete types, so it is easy to unit-test with mocks (this is the Dependency Inversion Principle of SOLID).
`RegisterAsync — email normalization + duplicate check`	The email is trimmed and lower-cased so 'A@x.com' and 'a@x.com ' collide, then EmailExistsAsync prevents duplicates by throwing a domain-level AuthException (which the controller turns into HTTP 409 Conflict).
`_hasher.Hash(req.Password)`	The plaintext password is never stored — only its BCrypt hash. BCrypt salts automatically and is deliberately slow to resist brute-force.
`_audit.RecordAsync(...)`	Every sensitive action (register/login/password-change) writes an immutable audit entry — essential for a regulated clinical-trial system.

`LoginAsync — verify + IsActive`	Verifies the password against the stored hash; the SAME generic 'Invalid email or password' message is returned whether the email is unknown or the password is wrong, so attackers can't enumerate valid emails. A blocked (IsActive=false) account is rejected separately.
`BuildAuthResponse`	Combines the freshly generated token, its expiry and a small SessionUser projection into the response the Angular client stores.

Authentication.API/Controllers/AuthController.cs (patient self-registration saga)

PURPOSE Exposes /api/auth login, admin-register, patient self-register and change-password. The patient-registration endpoint is a SAGA spanning two services with a compensating rollback.

```
[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _auth;
    private readonly IUserService _users;
    private readonly IHttpClientFactory _httpFactory;
    private readonly ILogger<AuthController> _logger;
    // ...ctor...

    [HttpPost("login")]
    [AllowAnonymous]
    public async Task<ActionResult<AuthResponse>> Login([FromBody] LoginRequest req)
    {
        try { return Ok(await _auth.LoginAsync(req)); }
        catch (AuthException ex) { return Unauthorized(new { error = ex.Message }); }
    }

    [HttpPost("register")]
    [RoleAuthorize(RolesEnum.Admin)]
    public async Task<IActionResult> Register([FromBody] RegisterRequest req)
    {
        if (req.RoleID == (int)RolesEnum.Patient)
            return BadRequest(new { error = "Patients must self-register via /api/auth/register/patient." });
        try { await _auth.RegisterAsync(req); _users.InvalidateListCache();
              return StatusCode(StatusCodes.Status201Created); }
        catch (AuthException ex) { return Conflict(new { error = ex.Message }); }
    }

    [HttpPost("register/patient")]
    [AllowAnonymous]
    public async Task<ActionResult<AuthResponse>> RegisterPatient([FromBody] PatientRegisterRequest req)
    {
        // Step 1: create the user account
        UserResponse createdUser;
        try { createdUser = await _auth.RegisterAsync(new RegisterRequest {
                  Name = req.Name, Email = req.Email, Password = req.Password,
                  Phone = req.Phone, RoleID = (int)RolesEnum.Patient }); }
        catch (AuthException ex) { return Conflict(new { error = ex.Message }); }

        // Step 2: auto-login to obtain the patient's JWT
        AuthResponse authResponse;
        try { authResponse = await _auth.LoginAsync(
                  new LoginRequest { Email = req.Email, Password = req.Password }); }
        catch (Exception ex) {
            _logger.LogError(ex, "Auto-login failed. Rolling back.");
            await TryDeleteUserAsync(createdUser.UserID);          // compensating action
            return StatusCode(500, new { error = "Account created but sign-in failed." });
        }

        // Step 3: create the Patient record in Patient.API
        try {
            var client = _httpFactory.CreateClient("PatientApi");
            client.DefaultRequestHeaders.Authorization =
                new AuthenticationHeaderValue("Bearer", authResponse.Token);   // token relay
            var patientBody = new { userID = createdUser.UserID, name = req.Name,
```

```
                dob = req.DOB, contactInfo = req.Phone ?? req.Email,
                enrollmentStatus = "Screening" };
            var json = JsonSerializer.Serialize(patientBody,
                new JsonSerializerOptions { PropertyNamingPolicy = JsonNamingPolicy.CamelCase });
            var content = new StringContent(json, Encoding.UTF8, "application/json");
            var response = await client.PostAsync("api/patients", content);
            if (!response.IsSuccessStatusCode) {
                await TryDeleteUserAsync(createdUser.UserID);      // compensating action
                return StatusCode(502, new { error = "Failed to create patient record." });
            }
        }
        catch (Exception ex) {
            _logger.LogError(ex, "Patient API unreachable. Rolling back.");
            await TryDeleteUserAsync(createdUser.UserID);          // compensating action
            return StatusCode(503, new { error = "Patient service is currently unavailable." });
        }

        // Step 4: return the token so the patient is immediately signed in
        _users.InvalidateListCache();
        return StatusCode(StatusCodes.Status201Created, authResponse);
    }

    private async Task TryDeleteUserAsync(long userId)
    {
        try { await _users.DeleteAsync(userId, userId); }
        catch (Exception ex) { _logger.LogWarning(ex, "Compensating delete failed."); }
    }
}
```

Line-by-line / block-by-block explanation

`[ApiController] + [Route("api/auth")]`	[ApiController] enables automatic model validation and binding-source inference; the route attribute prefixes every action with /api/auth.
`[AllowAnonymous] on Login`	Bypasses the global JwtAuthFilter — you obviously can't require a token to log in.
`Login try/catch AuthException`	A wrong password throws AuthException, which is translated to 401 Unauthorized with a clean error body.
`[RoleAuthorize(RolesEnum.Admin)] on Register`	Only Admins may create non-patient accounts. The typed attribute is the project's compile-safe replacement for [Authorize(Roles="Admin")]. It also blocks creating Patients here — they must self-register.
`RegisterPatient — Step 1`	Creates the User account. On a duplicate email it returns 409 immediately, before any other service is touched.
`Step 2 — auto-login`	Logs the new user in to obtain a JWT. If this fails, the just-created user is deleted (compensating action) and a 500 is returned — the system is left consistent.
`Step 3 — token relay to Patient.API`	A named HttpClient calls Patient.API, forwarding the patient's own JWT in the Authorization header (token relay) so the downstream service authorizes the call. On any non-2xx OR a network exception, the user is rolled back and 502/503 is returned.
`Step 4 — return token`	On full success the JWT is returned in the 201 body so the Angular RegisterComponent can log the patient straight in.
`TryDeleteUserAsync (compensating action)`	The saga's rollback. It is best-effort: if even the rollback fails it only logs a warning, flagging that manual cleanup may be needed. This is the Saga pattern — a sequence of local transactions, each with a compensating undo, used because a single ACID transaction can't span two databases/services.

Gateway.API/ocelot.json (excerpt)

PURPOSE Ocelot's routing table. Each route maps an upstream path (what Angular calls) to a downstream path + host + port (the real microservice). This is what makes the gateway a reverse proxy and single entry point.

```
{
  "Routes": [
    {
      "UpstreamPathTemplate": "/gateway/auth/{everything}",
      "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete", "Patch", "Options" ],
      "DownstreamPathTemplate": "/api/auth/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5033 } ]
    },
    {
      "UpstreamPathTemplate": "/gateway/adverse-events/{everything}",
      "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete", "Patch", "Options" ],
      "DownstreamPathTemplate": "/api/adverse-events/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5159 } ]
    },
    {
      "UpstreamPathTemplate": "/gateway/documents/{everything}",
      "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete", "Patch", "Options" ],
      "DownstreamPathTemplate": "/api/documents/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5232 } ]
    },
    {
      "UpstreamPathTemplate": "/gateway/kpi-reports/{everything}",
      "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete", "Patch", "Options" ],
      "DownstreamPathTemplate": "/api/kpi-reports/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5062 } ]
    }
    // ... routes for protocols(5193), patients/visits/enrollments(5095),
    //     deviations(5159), notifications(5210), audit-entries, users ...
  ],
  "GlobalConfiguration": { "BaseUrl": "http://localhost:5046" }
}
```

Line-by-line / block-by-block explanation

`UpstreamPathTemplate`	The public URL the Angular app hits, e.g. /gateway/adverse-events/5. The {everything} placeholder forwards any sub-path and query string.
`UpstreamHttpMethod (incl. Options)`	Which verbs the route accepts. OPTIONS is listed so CORS pre-flight passes through the gateway to the service.
`DownstreamPathTemplate + Host + Port`	Ocelot rewrites the URL to the internal /api/... path and forwards it to the correct service port (adverse-events -> 5159, documents -> 5232, kpi-reports -> 5062). The client never sees these ports — the gateway hides the topology (facade pattern).
`GlobalConfiguration.BaseUrl`	Declares the gateway's own external URL (5046). Angular's environment.apiUrl points here, so the frontend only ever knows ONE address.

Gateway.API/Program.cs

PURPOSE Bootstraps Ocelot from ocelot.json and applies a credentialed CORS policy, then runs the gateway.

```
using Ocelot.DependencyInjection;
using Ocelot.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Configuration.AddJsonFile("ocelot.json", optional: false, reloadOnChange: true);
builder.Services.AddOcelot(builder.Configuration);

builder.Services.AddCors(options =>
{
    options.AddPolicy("AngularDevClient", policy =>
```

```
        policy.WithOrigins("http://localhost:4200", "http://127.0.0.1:4200",
                           "http://localhost:62536", "http://127.0.0.1:62536" /* ... */)
              .WithHeaders("Authorization", "Content-Type", "Accept", "X-Requested-With")
              .WithMethods("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS")
              .AllowCredentials());
});

var app = builder.Build();
app.UseCors("AngularDevClient");
await app.UseOcelot();
app.Run();
```

Line-by-line / block-by-block explanation

`AddJsonFile("ocelot.json", reloadOnChange:true)`	Loads the routing table as configuration; reloadOnChange means edits are picked up without a restart.
`AddOcelot(...)`	Registers the Ocelot gateway services using that configuration.
`CORS with AllowCredentials()`	Unlike the individual services, the GATEWAY allows credentials — needed if cookies/credentialed requests flow through it. That's why origins are an explicit list (AllowAnyOrigin is illegal together with AllowCredentials).
`await app.UseOcelot()`	Inserts the Ocelot middleware that performs the actual request routing/proxying. It's the terminal middleware — requests are forwarded downstream from here.

AdverseEvent.API/Program.cs

PURPOSE Composition root for the Adverse-Event service. Almost identical structure to Authentication.API but pointed at ClinicalDb and registering the AE/Deviation repositories and services. Note it does NOT seed or expose HTTP clients.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers(o =>
{
    o.Filters.Add<JwtAuthFilter>();
    o.Filters.Add<ActivityLogFilter>();
    o.Filters.Add<DomainExceptionFilter>();
    o.Filters.Add<ValidateModelFilter>();
})
.AddJsonOptions(o => o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));

builder.Services.AddSwaggerGen(c => { /* same Bearer setup */ });
builder.Services.AddHttpContextAccessor();

builder.Services.AddDbContext<AdverseEventDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("ClinicalDb")));

builder.Services.AddMemoryCache();

builder.Services.AddScoped<IAdverseEventRepository, AdverseEventRepository>();
builder.Services.AddScoped<IDeviationRepository, DeviationRepository>();
builder.Services.AddScoped<IAdverseEventService, AdverseEventService>();
builder.Services.AddScoped<IDeviationService, DeviationService>();

var app = builder.Build();
app.UseGlobalExceptionHandler();
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(/*...*/); }
app.MapControllers();
app.Run();
```

Line-by-line / block-by-block explanation

`Same four global filters`	The service reuses the identical Shared.CL filter pipeline — proof of the 'microservice chassis' idea: cross-cutting behaviour lives once in Shared.CL and every service opts in with the same four lines.
`ClinicalDb connection`	Adverse events are clinical/operational data, so this service uses ClinicalDb (shared with Protocol, Patient and Analytics) rather than GovernanceDb.
`Two repositories + two services`	AdverseEvent.API owns two related aggregates — AdverseEventRecord and Deviation — grouped in one service because they belong to the same bounded context (avoids nano-services).
`No CORS / no seeding here`	Only services Angular calls directly through the gateway need their own CORS; and only Authentication seeds data. This service is reached via the gateway, which holds the CORS policy.

AdverseEvent.API/Controllers/AdverseEventsController.cs

PURPOSE REST endpoints for adverse events: paged+filtered list, get-by-id, role-restricted create and update. There is intentionally NO delete (clinical records are retained).

```
[ApiController]
[Route("api/adverse-events")]
public class AdverseEventsController : ControllerBase
{
    private readonly IAdverseEventService _svc;
    public AdverseEventsController(IAdverseEventService svc) => _svc = svc;

    [HttpGet]
    public async Task<ActionResult<PagedResult<AdverseEventResponse>>> List(
        [FromQuery] long? protocolId, [FromQuery] long? patientId,
        [FromQuery] string? severity, [FromQuery] string? status,
        [FromQuery] int page = 1, [FromQuery] int pageSize = 20)
    {
        if (page < 1) page = 1;
        if (pageSize is <= 0 or > 200) pageSize = 20;
        return Ok(await _svc.ListAsync(protocolId, patientId, severity, status, page, pageSize));
    }

    [HttpGet("{id:long}")]
    public async Task<ActionResult<AdverseEventResponse>> Get(long id)
    {
        var result = await _svc.GetAsync(id);
        return result is null ? NotFound() : Ok(result);
    }

    [HttpPost]
    [RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager,
                   RolesEnum.Investigator, RolesEnum.DataManager)]
    public async Task<ActionResult> Create([FromBody] CreateAdverseEventRequest req)
    {
        try { var eventID = await _svc.CreateAsync(req);
              return StatusCode(201, new { eventID }); }
        catch (DomainException ex) { return BadRequest(new { error = ex.Message }); }
    }

    [HttpPut("{id:long}")]
    [RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager,
                   RolesEnum.RegulatoryOfficer, RolesEnum.DataManager)]
    public async Task<ActionResult> Update(long id, [FromBody] UpdateAdverseEventRequest req)
    {
        try {
            var updated = await _svc.UpdateAsync(id, req);
            if (updated is null) return NotFound();
            return NoContent();   // client already has the new values
        }
        catch (DomainException ex) { return BadRequest(new { error = ex.Message }); }
```

```
    }
}
```

Line-by-line / block-by-block explanation

`Constructor depends on IAdverseEventService`	The controller is thin — it only orchestrates HTTP. All business logic lives behind the service interface, which keeps controllers testable and the layers clean.
`List with [FromQuery] params`	All filters (protocolId, patientId, severity, status) and paging (page, pageSize) arrive as query-string values. The controller clamps page>=1 and 1<=pageSize<=200 to stop a client from requesting a million rows.
`ActionResult<PagedResult<...>>`	Returns a typed paged envelope (Items + TotalCount + Page + PageSize). The strong type lets Swagger document the exact response schema.
`Get with {id:long}`	The :long route constraint guarantees the segment parses as a long, so 'abc' never reaches the action. A miss returns 404 NotFound.
`Create [RoleAuthorize(...4 roles)]`	Only Admin, CTM, Investigator and DataManager can REPORT an adverse event. On a business-rule violation the service throws DomainException -> 400.
`Update [RoleAuthorize(...)] returns NoContent`	A DIFFERENT role set may update (note RegulatoryOfficer can update but not create). On success it returns 204 No Content because the client already holds the values it just sent — this is the idempotent nature of PUT.
`No Delete action`	Deliberately absent: in a regulated trial, adverse events must be retained and audited, never hard-deleted.

DocumentCompliance.API/Program.cs (delta vs others) + Models/Document.cs

PURPOSE DocumentCompliance shares the same chassis but adds a warning suppression, and its Document model shows the data-annotation approach to schema + validation.

```
// Program.cs — note the GovernanceDb + warning suppression
builder.Services.AddDbContext<DocumentDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("GovernanceDb"))
           .ConfigureWarnings(w =>
               w.Ignore(RelationalEventId.PendingModelChangesWarning)));

// Models/Document.cs
[Table("Documents")]
public class Document
{
    [Key] public long DocumentID { get; set; }
    [Required, MaxLength(300)] public string Title { get; set; } = string.Empty;
    [Required, MaxLength(100)] public string Category { get; set; } = "Other";   // 17 eTMF categories
    public long? ProtocolID { get; set; }
    [Required, MaxLength(20)]  public string Version    { get; set; } = "1.0";
    [Required] public long     UploadedBy  { get; set; }
    [Required] public DateTime UploadedAt  { get; set; } = DateTime.UtcNow;
    [Required, MaxLength(50)]  public string Status     { get; set; } = "Draft";
    [MaxLength(1000)] public string? Notes { get; set; }
    [MaxLength(2000)] public string? DocumentLink { get; set; }   // external Drive/OneDrive URL
}
```

Line-by-line / block-by-block explanation

`GovernanceDb`	Documents are governance/regulatory artefacts, so this service shares GovernanceDb with Authentication.
`ConfigureWarnings(Ignore(PendingModelChangesWarning))`	Suppresses EF Core's noisy 'model has pending changes' warning that appears during active development when the snapshot and migrations are briefly out of sync.
`[Table("Documents")] / [Key]`	Maps the class to the Documents table and marks DocumentID as the primary key.

`[Required, MaxLength(n)]`	Drive both the generated SQL column constraints AND request validation. If a client posts a Title longer than 300 chars, ValidateModelFilter returns 422 before the code runs.
`nullable string? Notes / DocumentLink`	The ? marks optional columns; DocumentLink stores an external file URL rather than the file bytes — keeping the DB small.
`Status default 'Draft'`	New documents start as Draft; the supersession workflow later flips older versions to 'Superseded' when a new version is uploaded.

Shared.CL/Filters/JwtAuthFilter.cs

PURPOSE The shared global authorization filter that validates the JWT on every request in every service. It replaces the usual AddAuthentication().AddJwtBearer() middleware.

```
public class JwtAuthFilter : IAsyncAuthorizationFilter
{
    private readonly IConfiguration _config;
    public JwtAuthFilter(IConfiguration config) => _config = config;

    public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
    {
        // Skip endpoints marked [AllowAnonymous]
        bool allowAnonymous = context.ActionDescriptor.EndpointMetadata
            .Any(em => em.GetType() == typeof(AllowAnonymousAttribute));
        if (allowAnonymous) return;

        string? authHeader = context.HttpContext.Request.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(authHeader) ||
            !authHeader.StartsWith("Bearer ", StringComparison.OrdinalIgnoreCase))
        {
            context.Result = new ObjectResult(
                ApiResponse<object>.Fail("Unauthorized. Missing or invalid token.")) { StatusCode = 401 };
            return;
        }

        string token = authHeader.Substring("Bearer ".Length).Trim();
        if (string.IsNullOrWhiteSpace(token))
        {
            context.Result = new ObjectResult(
                ApiResponse<object>.Fail("Unauthorized. Empty bearer token.")) { StatusCode = 401 };
            return;
        }

        try
        {
            var tokenHandler = new JwtSecurityTokenHandler();
            string secretKey = Environment.GetEnvironmentVariable("JWT_SECRET")
                ?? _config["Jwt:Secret"]
                ?? throw new InvalidOperationException("Jwt:Secret is not configured.");
            string? issuer   = _config["Jwt:Issuer"];
            string? audience = _config["Jwt:Audience"];

            var validationParameters = new TokenValidationParameters
            {
                ValidateIssuerSigningKey = true,
                IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey)),
                ValidateIssuer   = !string.IsNullOrWhiteSpace(issuer),  ValidIssuer = issuer,
                ValidateAudience = !string.IsNullOrWhiteSpace(audience), ValidAudience = audience,
                ValidateLifetime = true,
                ClockSkew = TimeSpan.FromSeconds(30)
            };

            ClaimsPrincipal principal = tokenHandler.ValidateToken(token, validationParameters, out _);
            context.HttpContext.User = principal;
        }
        catch (Exception ex)
        {
            context.Result = new ObjectResult(
```

```
                ApiResponse<object>.Fail("Unauthorized. Token expired or invalid. " + ex.Message))
            { StatusCode = 401 };
        }
        await Task.CompletedTask;
    }
}
```

Line-by-line / block-by-block explanation

`IAsyncAuthorizationFilter`	Runs early in the MVC pipeline, before the action executes. Registered globally in every service's Program.cs, so EVERY endpoint is protected by default.
`AllowAnonymous skip`	It inspects endpoint metadata and bails out for [AllowAnonymous] endpoints like login — so those remain reachable without a token.
`Bearer header extraction (defensive)`	It carefully checks the header exists and starts with 'Bearer ', and that the token isn't blank — returning 401 with a clean ApiResponse body rather than throwing.
`secret = JWT_SECRET env var ?? config`	Prefers the JWT_SECRET environment variable (production) and falls back to appsettings (dev). This is the 12-factor 'config in the environment' principle and keeps secrets out of source control in production.
`TokenValidationParameters`	Validates the signature (with the symmetric key), optionally the issuer/audience, and the lifetime, allowing a 30-second ClockSkew so tiny clock differences between machines don't reject valid tokens.
`context.HttpContext.User = principal`	On success the validated ClaimsPrincipal is attached to the request so downstream code (RoleAuthorizeFilter, controllers) can read the user id and role. Because each service does this independently, authentication is decentralized — no service has to call the auth service (zero-trust + stateless).

Shared.CL/Filters/RoleAuthorizeFilter.cs (+ attribute)

PURPOSE Enforces role-based access using the typed RolesEnum. A custom attribute [RoleAuthorize(...)] wires the filter onto a controller/action.

```
public class RoleAuthorizeFilter : IAsyncAuthorizationFilter
{
    private readonly RolesEnum[] _allowedRoles;
    public RoleAuthorizeFilter(RolesEnum[] roles) => _allowedRoles = roles;

    public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
    {
        bool allowAnonymous = context.ActionDescriptor.EndpointMetadata
            .Any(em => em.GetType() == typeof(AllowAnonymousAttribute));
        if (allowAnonymous) return;

        if (context.HttpContext.User.Identity?.IsAuthenticated != true)
        {
            context.Result = new ObjectResult(
                ApiResponse<object>.Fail("Unauthorized. Please log in.")) { StatusCode = 401 };
            return;
        }

        string? userRole = context.HttpContext.User.FindFirstValue(ClaimTypes.Role);
        bool hasRole = _allowedRoles.Any(r =>
            string.Equals(r.ToString(), userRole, StringComparison.OrdinalIgnoreCase));

        if (!hasRole)
            context.Result = new ObjectResult(
                ApiResponse<object>.Fail("Forbidden. You do not have permission.")) { StatusCode = 403 };

        await Task.CompletedTask;
    }
}
```

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method, AllowMultiple = false)]
public class RoleAuthorizeAttribute : TypeFilterAttribute
{
    public RoleAuthorizeAttribute(params RolesEnum[] roles) : base(typeof(RoleAuthorizeFilter))
        => Arguments = new object[] { roles };
}
```

Line-by-line / block-by-block explanation

`Runs AFTER JwtAuthFilter`	By the time this filter runs, JwtAuthFilter has already attached the principal. This filter only does authorization (what you may do), not authentication (who you are).
`401 vs 403 distinction`	Not authenticated -> 401 Unauthorized. Authenticated but wrong role -> 403 Forbidden. This is the textbook-correct status-code split.
`FindFirstValue(ClaimTypes.Role)`	Reads the Role claim that JwtTokenService embedded, and case-insensitively checks it against the allowed roles.
`RoleAuthorizeAttribute : TypeFilterAttribute`	TypeFilterAttribute lets the filter receive constructor arguments (the roles array) while still being resolved from DI. The result is the clean, compile-checked usage [RoleAuthorize(RolesEnum.Admin, RolesEnum.ClinicalTrialManager)] seen on the controllers — far safer than magic strings in [Authorize(Roles="...")].

Shared.CL/Middleware/GlobalExceptionMiddleware.cs

PURPOSE Terminal middleware that catches any unhandled exception escaping the MVC pipeline and returns a consistent ErrorDetail JSON, mapping exception types to status codes.

```
public sealed class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _logger;
    private static readonly JsonSerializerOptions _jsonOptions =
        new() { PropertyNamingPolicy = JsonNamingPolicy.CamelCase };

    public GlobalExceptionMiddleware(RequestDelegate next, ILogger<GlobalExceptionMiddleware> logger)
    { _next = next; _logger = logger; }

    public async Task InvokeAsync(HttpContext context)
    {
        try { await _next(context); }
        catch (Exception ex) { await HandleExceptionAsync(context, ex); }
    }

    private async Task HandleExceptionAsync(HttpContext context, Exception ex)
    {
        int statusCode; string errorTitle;
        switch (ex)
        {
            case DomainException:
                statusCode = StatusCodes.Status400BadRequest;  errorTitle = "Business Rule Violation";
                _logger.LogWarning(ex, "Domain exception: {Message}", ex.Message); break;
            case NotFoundException:
                statusCode = StatusCodes.Status404NotFound;     errorTitle = "Not Found";
                _logger.LogWarning(ex, "Not found: {Message}", ex.Message); break;
            case UnauthorizedAccessException:
                statusCode = StatusCodes.Status401Unauthorized; errorTitle = "Unauthorized";
                _logger.LogWarning(ex, "Unauthorized access attempt."); break;
            default:
                statusCode = StatusCodes.Status500InternalServerError;
                errorTitle = "An unexpected error occurred";
                _logger.LogError(ex, "Unhandled exception."); break;
        }

        var payload = new ErrorDetail
        {
            StatusCode = statusCode, Error = errorTitle,
```

```
            Details = statusCode < 500 ? [ex.Message] : [],   // hide internals on 500
            TraceId = context.TraceIdentifier
        };
        context.Response.ContentType = "application/json";
        context.Response.StatusCode  = statusCode;
        await context.Response.WriteAsync(JsonSerializer.Serialize(payload, _jsonOptions));
    }
}
```

Line-by-line / block-by-block explanation

`RequestDelegate _next`	Middleware receives the next component in the pipeline. Calling _next(context) passes control downward; wrapping it in try/catch means this component sees any exception thrown anywhere below it.
`switch on exception type`	DomainException -> 400, NotFoundException -> 404, UnauthorizedAccessException -> 401, anything else -> 500. Each is logged at the appropriate level (Warning for expected, Error for unexpected).
`Details hidden on 500`	For server errors (>=500) the raw message is NOT returned to the client — only logged — so internal details/stack info don't leak. For 4xx the message is safe to surface.
`TraceId = context.TraceIdentifier`	Every error response carries a trace id, so a user-reported error can be correlated with the server logs — the seed of distributed tracing.
`Registered first via UseGlobalExceptionHandler()`	Because it is added first in Program.cs, it is the outermost wrapper around the whole pipeline — the safety net that guarantees clients always get structured JSON, never an HTML stack-trace page.

Lifetrack.UI/src/app/app.module.ts

PURPOSE The root Angular module. It bootstraps AppComponent, imports the router + HttpClient, and registers the AuthInterceptor and a global IST date default.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';
import { DATE_PIPE_DEFAULT_OPTIONS } from '@angular/common';
import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { AuthInterceptor } from './core/interceptors/auth.interceptor';

@NgModule({
  declarations: [ AppComponent ],
  imports: [ BrowserModule, HttpClientModule, AppRoutingModule ],
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
    { provide: DATE_PIPE_DEFAULT_OPTIONS, useValue: { timezone: '+0530' } }  // IST
  ],
  bootstrap: [ AppComponent ]
})
export class AppModule {}
```

Line-by-line / block-by-block explanation

`declarations / imports / providers / bootstrap`	declarations lists components owned by this module (AppComponent); imports pulls in BrowserModule, HttpClientModule and the routing module; providers registers app-wide services; bootstrap names the root component Angular renders first.
`HTTP_INTERCEPTORS, multi: true`	Registers AuthInterceptor into the interceptor chain. multi:true means it's added to an array (several interceptors can co-exist) rather than replacing the token.
`DATE_PIPE_DEFAULT_OPTIONS timezone +0530`	Globally configures Angular's DatePipe to render in IST (UTC+5:30), so every {{ date | date }} shows Indian time without per-usage configuration.

Lifetrack.UI/src/app/app-routing.module.ts

PURPOSE Top-level routes. /auth and /dashboard are lazy-loaded feature modules; /dashboard is protected by AuthGuard; unknown paths redirect to the dashboard.

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { AuthGuard } from './core/guards/auth.guard';

const routes: Routes = [
  { path: 'auth',
    loadChildren: () => import('./features/auth/auth.module').then(m => m.AuthModule) },
  { path: 'dashboard',
    loadChildren: () => import('./features/dashboard/dashboard.module').then(m => m.DashboardModule),
    canActivate: [AuthGuard] },                      // must be logged in
  { path: '',   redirectTo: 'dashboard', pathMatch: 'full' },
  { path: '**', redirectTo: 'dashboard' }
];

@NgModule({
  imports: [ RouterModule.forRoot(routes) ],
  exports: [ RouterModule ]
})
export class AppRoutingModule {}
```

Line-by-line / block-by-block explanation

`loadChildren: () => import(...)`	Lazy loading — the auth and dashboard bundles are only downloaded the first time the user visits those routes, shrinking the initial load.
`canActivate: [AuthGuard]`	The dashboard branch is guarded; AuthGuard runs before activation and redirects to /auth/login if no valid token exists.
`redirectTo dashboard (empty + **)`	The empty path and the wildcard both send users to /dashboard; with the guard in place, unauthenticated users bounce to login.
`RouterModule.forRoot(routes)`	forRoot is used exactly once at the app root to configure the router with these top-level routes (feature modules use forChild).

Lifetrack.UI/src/app/core/services/auth.service.ts

PURPOSE The single source of truth for authentication on the client: login/logout, token storage, and — importantly — reading role/userId/expiry directly from the JWT payload rather than the tamperable cached user object.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly TOKEN_KEY = 'lt_token';
  private readonly USER_KEY  = 'lt_user';
  private readonly DOTNET_ROLE_CLAIM =
    'http://schemas.microsoft.com/ws/2008/06/identity/claims/role';

  private currentUserSubject = new BehaviorSubject<UserInfo | null>(this.getStoredUser());
  currentUser$ = this.currentUserSubject.asObservable();

  constructor(private http: HttpClient, private router: Router) {}

  login(req: LoginRequest): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${environment.apiUrl}/auth/login`, req).pipe(
      tap(res => {
        localStorage.setItem(this.TOKEN_KEY, res.token);
        localStorage.setItem(this.USER_KEY, JSON.stringify(res.user));
        this.currentUserSubject.next(res.user);
      })
    );
  }
```

```
  handleAuthResponse(res: AuthResponse): void {        // used after patient self-register
    localStorage.setItem(this.TOKEN_KEY, res.token);
    localStorage.setItem(this.USER_KEY, JSON.stringify(res.user));
    this.currentUserSubject.next(res.user);
  }

  logout(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    localStorage.removeItem(this.USER_KEY);
    this.currentUserSubject.next(null);
    this.router.navigate(['/auth/login']);
  }

  getToken(): string | null { return localStorage.getItem(this.TOKEN_KEY); }

  isLoggedIn(): boolean {
    const token = this.getToken();
    if (!token) return false;
    const exp = this.getTokenExpiry();
    return exp ? exp > Date.now() : true;
  }

  // Reads claims straight from the JWT payload — the cached user object is tamperable.
  private decodeToken(): Record<string, any> | null {
    const token = this.getToken();
    if (!token) return null;
    try {
      const parts = token.split('.');
      if (parts.length !== 3) return null;
      const payload = parts[1].replace(/-/g, '+').replace(/_/g, '/');
      const padded  = payload + '='.repeat((4 - payload.length % 4) % 4);
      return JSON.parse(atob(padded));
    } catch { return null; }
  }

  getRoleFromToken(): string | null {
    const claims = this.decodeToken();
    return claims ? (claims['role'] ?? claims[this.DOTNET_ROLE_CLAIM] ?? null) : null;
  }

  getTokenExpiry(): number | null {
    const claims = this.decodeToken();
    if (!claims?.['exp']) return null;
    return Number(claims['exp']) * 1000;   // JWT exp is seconds; JS wants ms
  }
}
```

Line-by-line / block-by-block explanation

`@Injectable({ providedIn: 'root' })`	Registers AuthService as an application-wide singleton — every component and guard shares the same instance and the same currentUser$ stream.
`BehaviorSubject + currentUser$`	A BehaviorSubject holds the current user and replays it to any new subscriber immediately, so components that load later still get the current login state. .asObservable() exposes a read-only view.
`login().pipe(tap(...))`	Posts credentials, and the tap side-effect stores the token + user in localStorage and pushes the user into the subject — all without altering the emitted value.
`handleAuthResponse`	Reused after patient self-registration, where the 201 response already contains a token, so the patient is logged in without a second round-trip.
`logout()`	Clears storage, emits null and navigates to login.
`isLoggedIn() checks expiry`	Not just 'is there a token' but 'is it still valid' — it compares the exp claim (converted to ms) against now.

`decodeToken() base64url`	Manually base64url-decodes the JWT payload (replacing -/_ and re-padding) and JSON-parses it. No signature check here — that's the backend's job; the client only reads claims.
`getRoleFromToken — security note`	Role is read from the SIGNED token, NOT from the lt_user object, because a user could edit localStorage in DevTools to fake 'Admin'. Since the backend re-validates the signature on every request, the token claims are authoritative; the guard is just UX.

Lifetrack.UI/src/app/core/interceptors/auth.interceptor.ts

PURPOSE Attaches the Bearer token to every outgoing request and centrally handles 401s (session expiry) with a guard against a stampede of parallel logouts.

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  private static logoutInFlight = false;
  constructor(private auth: AuthService, private router: Router) {}

  intercept(req: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
    const token = this.auth.getToken();
    const cloned = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;

    return next.handle(cloned).pipe(
      catchError((err: HttpErrorResponse) => {
        if (err.status === 401) {
          // 401 from login = wrong credentials -> let the component show it
          if (req.url.includes('/auth/login')) return throwError(() => err);

          // Otherwise the token expired: clear session once, even if 6 parallel
          // requests all 401 at the same time (static latch prevents a router race).
          if (!AuthInterceptor.logoutInFlight) {
            AuthInterceptor.logoutInFlight = true;
            this.auth.logout();
            queueMicrotask(() => { AuthInterceptor.logoutInFlight = false; });
          }
          return EMPTY;   // swallow — the user is being logged out anyway
        }
        return throwError(() => err);
      })
    );
  }
}
```

Line-by-line / block-by-block explanation

`req.clone({ setHeaders: Authorization })`	HttpRequest objects are immutable, so the interceptor clones the request and adds the 'Authorization: Bearer <token>' header. Every service call is therefore authenticated automatically — no component sets the header itself.
`catchError on 401`	Centralized response handling: a 401 anywhere means the session is gone.
`login 401 passthrough`	A 401 specifically from /auth/login is a wrong-password case, not an expiry, so it's re-thrown for the LoginComponent to display.
`static logoutInFlight latch`	When the dashboard fires 6 parallel requests (e.g. via forkJoin) and they ALL get 401, without a guard each would call logout() and they'd race the router. The static boolean ensures logout runs once; queueMicrotask resets it after the navigation dispatches.
`return EMPTY`	Swallows the error for non-login 401s because the user is already being redirected — there's no point letting each component try to render an error.

Lifetrack.UI/src/app/core/guards/auth.guard.ts + role.guard.ts

PURPOSE Two CanActivate guards. AuthGuard requires a valid session; RoleGuard additionally checks the JWT role against roles declared on the route's data.

```
// auth.guard.ts
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(): boolean {
    if (this.auth.isLoggedIn()) return true;
    this.router.navigate(['/auth/login']);
    return false;
  }
}

// role.guard.ts
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(route: ActivatedRouteSnapshot): boolean {
    if (!this.auth.isLoggedIn()) { this.router.navigate(['/auth/login']); return false; }

    const allowed: string[] = route.data?.['roles'] ?? [];
    if (allowed.length === 0) return true;                // no restriction

    const tokenRole = this.auth.getRoleFromToken();
    const userRole  = tokenRole ?? this.auth.currentUser?.role ?? '';
    if (allowed.includes(userRole)) return true;

    this.router.navigate(['/dashboard']);                 // wrong role -> role home
    return false;
  }
}

// Usage in a route:
//  { path: 'documents', component: CtmDocumentsPageComponent, canActivate: [RoleGuard],
//    data: { roles: ['ClinicalTrialManager', 'RegulatoryOfficer'], title: 'Documents' } }
```

Line-by-line / block-by-block explanation

`CanActivate returns boolean`	A guard returning true allows navigation, false blocks it. Both guards redirect on failure rather than just blocking, so the user lands somewhere sensible.
`AuthGuard`	The simplest gate — used on the whole /dashboard branch. If isLoggedIn() is false it bounces to login.
`RoleGuard reads route.data['roles']`	The allowed roles are declared declaratively on each child route's data object. An empty list means 'any authenticated user'.
`role from token, fallback to currentUser`	It prefers the JWT role (authoritative) and falls back to the cached user only if needed. A mismatched role redirects to /dashboard (the user's own home), not an error page.
`Comment: backend is still the source of truth`	The guard's own docstring stresses it only prevents the wrong UI from rendering — the server filters (JwtAuthFilter + RoleAuthorizeFilter) are the real security boundary.

Lifetrack.UI/.../features/auth/login/login.component.ts

PURPOSE A Reactive-Forms login screen: builds a validated FormGroup, blocks submit while invalid, shows a spinner via finalize, and maps backend errors to a friendly message.

```
@Component({ selector: 'app-login', standalone: false,
             templateUrl: './login.component.html', styleUrls: ['./login.component.css'] })
export class LoginComponent {
  form: FormGroup;
```

```
  loading = false; error = ''; showPass = false;

  constructor(private fb: FormBuilder, private auth: AuthService, private router: Router) {
    if (this.auth.isLoggedIn()) this.router.navigate(['/dashboard']);   // already in
    this.form = this.fb.group({
      email:    ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  get email()    { return this.form.get('email')!; }
  get password() { return this.form.get('password')!; }

  submit(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    this.loading = true; this.error = '';
    this.auth.login(this.form.value).pipe(
      finalize(() => this.loading = false)
    ).subscribe({
      next: () => this.router.navigate(['/dashboard']),
      error: err => {
        this.error = err.error?.errors?.[0] ?? err.error?.message ??
                     err.error?.error ?? 'Invalid email or password. Please try again.';
      }
    });
  }

  togglePass(): void { this.showPass = !this.showPass; }
}
```

Line-by-line / block-by-block explanation

`constructor redirect if logged in`	If a valid session already exists, the user is sent straight to the dashboard instead of seeing the login form again.
`fb.group({...validators...})`	FormBuilder creates a FormGroup with two controls. email requires a valid email; password requires at least 6 chars. Validators are plain functions, which makes them trivially unit-testable.
`get email()/password()`	Convenience getters so the template can read control state (errors, touched) concisely.
`submit guard + markAllAsTouched`	If the form is invalid, every control is marked touched so all error messages appear at once, and the method returns without calling the API.
`.pipe(finalize(() => loading=false))`	finalize runs whether the request succeeds OR errors, so the spinner always stops — a common source of stuck UIs is forgetting this.
`subscribe next/error`	On success it navigates to the dashboard; on error it digs through the possible backend error shapes (errors[]/message/error) and falls back to a generic message.

Lifetrack.UI/.../features/auth/register/register.component.ts

PURPOSE Patient self-registration with rich Reactive-Forms validation: a cross-field password-match validator, a custom date-of-birth validator (must be an adult, not in the future, not absurdly old) and an India-specific phone validator.

```
export class RegisterComponent {
  form: FormGroup;
  loading = false; error = ''; showPass = false; showPass2 = false;
  readonly today  = new Date().toISOString().split('T')[0];
  readonly minDob = new Date(new Date().setFullYear(new Date().getFullYear() - 120))
                      .toISOString().split('T')[0];

  constructor(private fb: FormBuilder, private http: HttpClient,
              private auth: AuthService, private router: Router) {
    if (this.auth.isLoggedIn()) this.router.navigate(['/dashboard']);
    this.form = this.fb.group({
      name:     ['', [Validators.required, Validators.minLength(2), Validators.maxLength(200)]],
      email:    ['', [Validators.required, Validators.email, Validators.maxLength(256)]],
```

```
      dob:      ['', [Validators.required, this.dobValidator.bind(this)]],
      password: ['', [Validators.required, Validators.minLength(8), Validators.maxLength(128)]],
      confirm:  ['', Validators.required],
      phone:    ['', [Validators.maxLength(32), this.phoneValidator]],
    }, { validators: this.passwordsMatch });          // group-level validator
  }

  private passwordsMatch(g: FormGroup) {
    const pw = g.get('password')?.value, c = g.get('confirm')?.value;
    return pw && c && pw !== c ? { mismatch: true } : null;
  }

  private dobValidator(control: AbstractControl): ValidationErrors | null {
    const value = control.value; if (!value) return null;
    const dob = new Date(value); const today = new Date(); today.setHours(0,0,0,0);
    if (dob >= today) return { dobFuture: true };
    const age18 = new Date(today); age18.setFullYear(today.getFullYear() - 18);
    if (dob > age18) return { dobMinAge: true };       // must be 18+ for trial consent
    const age120 = new Date(today); age120.setFullYear(today.getFullYear() - 120);
    if (dob < age120) return { dobMaxAge: true };
    return null;
  }

  private phoneValidator(control: AbstractControl): ValidationErrors | null {
    const value = control.value?.trim(); if (!value) return null;   // optional
    return /^[6-9]\d{9}$/.test(value) ? null : { phonePattern: true };  // Indian mobile
  }

  submit(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    this.loading = true; this.error = '';
    const v = this.form.value;
    const body: any = { name: v.name.trim(), email: v.email.trim(), dob: v.dob, password: v.password };
    if (v.phone?.trim()) body.phone = v.phone.trim();
    this.http.post<AuthResponse>(`${environment.apiUrl}/auth/register/patient`, body).subscribe({
      next: res => { this.auth.handleAuthResponse(res); this.router.navigate(['/dashboard']); },
      error: err => { this.loading = false;
        this.error = err?.error?.error ?? err?.error?.message ??
                     'Registration failed. This email may already be registered.'; }
    });
  }
}
```

Line-by-line / block-by-block explanation

`today / minDob readonly fields`	Pre-computed ISO date strings used as [max] and [min] on the date input so the native picker itself prevents future or >120-year-old dates — defence in depth alongside the validator.
`group-level { validators: passwordsMatch }`	Cross-field validation can't live on a single control, so passwordsMatch is attached to the FormGroup; it compares password and confirm and returns { mismatch:true } when they differ.
`dobValidator (custom)`	A custom validator returning typed error keys: dobFuture (DOB today/future), dobMinAge (under 18 — clinical trials need adult consent), dobMaxAge (over 120). Returning null means valid.
`phoneValidator (custom, optional)`	Skips validation when blank (the field is optional) and otherwise enforces an Indian mobile pattern: 10 digits starting 6-9.
`submit builds body conditionally`	Phone is only added to the payload if present, then POSTs to /auth/register/patient. On success it reuses auth.handleAuthResponse to log the patient in immediately.
`error mapping`	Surfaces the backend's error/message, defaulting to a duplicate-email hint — matching the 409 the AuthController returns.

Part I ASP.NET Core 8 Web API

RESTful APIs, EF Core, middleware, filters, JWT and the LifeTrack backend. · **101 questions**, each with an answer, project code where relevant, and a real-world analogy.

Q1 What is a Web API and how does it differ from a traditional MVC application?

A Web API returns data (JSON/XML) instead of HTML views. It uses HTTP verbs (GET/POST/PUT/DELETE) and is consumed by clients like Angular or mobile apps. In LifeTrack, Angular consumes 8 backend Web APIs via the Ocelot Gateway.

> **REAL-WORLD** Like a restaurant kitchen window: you pass an order slip (request) and get back food (data), never the recipe or the HTML menu.

Q2 Explain the difference between REST and SOAP with a LifeTrack context.

REST uses HTTP, is stateless, and returns JSON (as in all LifeTrack APIs). SOAP uses XML envelopes and WSDL contracts — heavier, but supports WS-Security. LifeTrack uses REST for all inter-service and client communication.

> **REAL-WORLD** REST is texting a waiter in plain language; SOAP is filling a formal multi-page government form (XML envelope) that must follow an exact template.

Q3 What are the six constraints of REST (Richardson Maturity Model)?

Uniform Interface, Stateless, Cacheable, Client-Server, Layered System, Code on Demand (optional). LifeTrack adheres to statelessness (JWT tokens per request), layered system (Gateway → services).

> **REAL-WORLD** Like the rules of the road: stateless = each car carries its own fuel, cacheable = you can photocopy a map, uniform = every junction works the same way.

Q4 What is the ASP.NET Core Web API project structure? Name key files.

Program.cs (app startup, DI registration), Controllers folder, Models, DTOs, appsettings.json, launchSettings.json. LifeTrack adds Repositories, Services, Data (DbContext), Enums, and Migrations folders per service.

> **REAL-WORLD** Like a house blueprint: Program.cs is the foundation, Controllers are the rooms, appsettings.json is the fuse box label.

Q5 What is the Minimal API approach in .NET 8? How is it different from controller-based?

Minimal APIs define routes in Program.cs without controllers using app.MapGet/Post. LifeTrack uses controller-based APIs (inheriting ControllerBase) for better separation and filter support.

> **REAL-WORLD** Minimal API is a food truck (everything at one counter); controller-based is a full restaurant with separate kitchen, waiters and menu.

Q6 What does WebApplication.CreateBuilder(args) do in Program.cs?

It creates a WebApplicationBuilder that configures the DI container, configuration sources (appsettings.json, env variables), logging, and Kestrel. LifeTrack's every Program.cs starts with this.

> **REAL-WORLD** Like switching on the mains before wiring a house — it powers the panel that everything else plugs into.

Q7 What is the difference between app.MapControllers() and app.UseEndpoints()?

In .NET 6+, app.MapControllers() is the simplified version. UseEndpoints is the older pre-6 pattern. Both register attribute-routed controller endpoints. LifeTrack uses MapControllers().

REAL-WORLD Like two editions of the same instruction manual — the newer one (MapControllers) just dropped the boilerplate cover page.

Q8 What HTTP status codes did you use in LifeTrack and when?

200 OK (GETs), 201 Created (POSTs), 204 NoContent (PUTs with no body), 400 BadRequest (validation), 401 Unauthorized, 404 NotFound (NotFoundException), 409 Conflict (duplicate email), 422 Unprocessable (ModelState errors).

REAL-WORLD Like a parcel tracking status: 200 'Delivered', 201 'Dispatched', 404 'Address not found', 409 'Duplicate order'.

Q9 Explain idempotency in REST. Which HTTP methods are idempotent?

Calling the same request N times has the same effect as calling once. GET, PUT, DELETE are idempotent. POST is not. LifeTrack's PUT /adverse-events/{id} is idempotent — sending the same update repeatedly yields the same state.

REAL-WORLD Pressing a lift button 5 times (idempotent) still brings one lift; placing an online order 5 times (POST) charges you 5 times.

Q10 What is JSON serialization? How did LifeTrack configure it?

Converting .NET objects to JSON and back. LifeTrack adds JsonStringEnumConverter so enum values like AESeverity.Mild serialize as "Mild" not 0. Configured via .AddJsonOptions() in Program.cs.

REAL-WORLD Like translating a Python list into a JSON shopping list so a phone app can read it — and back again.

Q11 What is [ApiController] attribute and what does it enable?

Enables automatic model validation (returns 400 if ModelState invalid), binding source inference ([FromBody], [FromQuery]), and problem details responses. All LifeTrack controllers use this.

REAL-WORLD Like a smart form that auto-checks your entries and rejects a blank required field before you even hit submit.

Q12 What is attribute routing? Give a LifeTrack example.

[Route("api/adverse-events")] on AdverseEventsController. [HttpGet("{id:long}")] creates GET /api/adverse-events/5. The `:long` constraint ensures id is a long integer.

REAL-WORLD Like a building's room-numbering scheme: api/adverse-events/5 always points to the same room.

Q13 What is the difference between [FromBody], [FromQuery], [FromRoute]?

[FromBody] reads JSON request body. [FromQuery] reads URL query strings. [FromRoute] reads route template segments. AdverseEventsController.List uses [FromQuery] for protocolId, severity, page, pageSize.

REAL-WORLD Like sorting incoming mail: body = the letter inside, query = notes on the envelope, route = the street address itself.

Q14 How did you implement pagination in LifeTrack?

Via query parameters: page and pageSize. AdverseEventsController clamps page ≥ 1 and pageSize between 1–200. The repository returns a PagedResult<T> with Items, TotalCount, Page, PageSize. Angular renders paginator UI.

> **REAL-WORLD** Like a book showing 20 results per page with 'Next' — you never dump all 10,000 entries on one screen.

Q15 What is ControllerBase vs Controller? Why use ControllerBase for APIs?

ControllerBase has no view support. Controller inherits ControllerBase and adds view-rendering methods (View(), PartialView()). APIs should use ControllerBase — LifeTrack does throughout.

> **REAL-WORLD** ControllerBase is a delivery-only kitchen; Controller also has a dining room (views) you don't need for an API.

Q16 What is the difference between IActionResult and ActionResult<T>?

IActionResult is untyped. ActionResult<T> provides type safety and enables Swagger to infer the response schema. LifeTrack uses ActionResult<PagedResult<AdverseEventResponse>> for typed responses.

> **REAL-WORLD** Like labelling a parcel with its exact contents so the courier's system knows what's inside without opening it.

Q17 How did you handle CRUD in the AdverseEvent API?

GET /api/adverse-events (list + filter), GET /api/adverse-events/{id} (single), POST (create, role-restricted), PUT /{id} (update). No DELETE — adverse events must be audited and retained in clinical trials.

> **REAL-WORLD** Like a filing cabinet: add a folder (POST), read one (GET id), read a drawer (GET list), update a folder (PUT) — but adverse events are never shredded (no DELETE).

Q18 What is model binding in ASP.NET Core?

Automatically maps HTTP request data (query strings, route values, request body) to action method parameters. LifeTrack relies on it for CreateAdverseEventRequest bound from [FromBody].

> **REAL-WORLD** Like a hotel check-in form that automatically fills your room preferences from the booking data.

Q19 Explain route constraints. Give an example from LifeTrack.

{id:long} ensures the route segment must parse as a long. Without it, "abc" would reach the action and cause a runtime exception. LifeTrack uses :long on all entity ID route params.

> **REAL-WORLD** Like a parking gate that only opens for number plates (long), refusing letters.

Q20 How did LifeTrack return consistent error responses across all APIs?

Via the shared DomainExceptionFilter (DomainException → 400) and GlobalExceptionMiddleware (unhandled → 500). Both return an ErrorDetail JSON object with StatusCode, Error, Details[], TraceId.

> **REAL-WORLD** Like every department using the same complaint form, so customers always get a reply in the same format.

Q21 What is Entity Framework Core Code-First? How did you use it?

Model classes define the schema; EF Core generates migrations and creates/updates the DB. LifeTrack has 8 DbContext classes, one per service (e.g., AdverseEventDbContext), each connected to SQL Server.

> **REAL-WORLD** Like designing furniture from a sketch first, then the factory builds it — your C# classes are the sketch, the DB is the furniture.

Q22 What is DbContext and what is it responsible for?

DbContext is the EF Core unit of work and identity map. It holds DbSet<T> properties, tracks entity changes, and translates LINQ to SQL. AdverseEventDbContext exposes DbSet<AdverseEventRecord> and DbSet<Deviation>.

> **REAL-WORLD** Like a librarian who tracks which books you've borrowed and files the changes when you return them.

Q23 What is a Migration in EF Core? How do you apply it?

A migration is a C# file describing a schema change (Add-Migration name / dotnet ef migrations add name). Apply with Update-Database or dotnet ef database update. LifeTrack has multiple migrations per service.

> **REAL-WORLD** Like a renovation plan you can apply or roll back — 'add a window on the north wall'.

Q24 What is the Repository Pattern? How is it used in LifeTrack?

Abstracts data access behind an interface. IAdverseEventRepository → AdverseEventRepository. Controllers depend only on IAdverseEventService; services depend on IAdverseEventRepository. Enables testability and swappable storage.

> **REAL-WORLD** Like a warehouse clerk: the rest of the staff just ask the clerk, never touch the shelves directly.

Q25 What is the difference between AsNoTracking() and tracked queries?

Tracked queries allow EF to detect and save changes. AsNoTracking() is faster for read-only queries — no change tracking overhead. LifeTrack list queries use AsNoTracking() where updates are not needed.

> **REAL-WORLD** Reading a noticeboard you won't edit (NoTracking) vs taking a form home to fill and return (tracked).

Q26 What is lazy loading vs eager loading in EF Core?

Lazy loading fetches related data on access (N+1 problem). Eager loading uses .Include() to fetch related data in one query. LifeTrack uses eager loading where navigation properties are needed (e.g., user role navigation in JwtTokenService).

> **REAL-WORLD** Eager = bringing the whole grocery bag at once; lazy = running back to the shop each time you need one item.

Q27 How did you implement server-side filtering in AdverseEventsController?

The repository builds an IQueryable<AdverseEventRecord> and applies .Where() conditions for protocolId, patientId, severity, status if the parameters are non-null. Then .Skip().Take() applies pagination before materializing.

> **REAL-WORLD** Like a search filter on a shopping site — tick 'Severe' and 'Open' and only matching items load.

Q28 **What are Data Annotations in EF Core? Give LifeTrack examples.**

[Key] (DocumentID), [Required] (Title), [MaxLength(300)] (Title), [Table("Documents")] (override table name). These control schema generation and also serve as model validation rules.

> **REAL-WORLD** Like sticky notes on a form field: 'Required', 'Max 300 chars' — they shape the form and check input.

Q29 **What is the difference between SaveChangesAsync and SaveChanges?**

SaveChangesAsync is the async non-blocking version. LifeTrack uses SaveChangesAsync throughout repositories to avoid blocking the ASP.NET Core thread pool.

> **REAL-WORLD** Like sending an email in the background so you can keep typing, vs the screen freezing until it sends.

Q30 **How did you handle database seeding in LifeTrack?**

DbSeeder.SeedAsync(db) is called in Program.cs after EnsureCreatedAsync(). It inserts default admin users, roles, and seed data if the table is empty. Uses if (!db.Users.Any()) pattern.

> **REAL-WORLD** Like pre-stocking a new shop with default products before opening day.

Q31 **What is Middleware in ASP.NET Core? How is it different from a Filter?**

Middleware operates on the HttpContext at the pipeline level (before MVC). Filters run within the MVC pipeline after routing and model binding. GlobalExceptionMiddleware is middleware; JwtAuthFilter and DomainExceptionFilter are filters.

> **REAL-WORLD** Middleware is airport security every passenger passes; a filter is the gate check only for boarding passengers.

Q32 **Explain the ASP.NET Core middleware pipeline. What is the "next" delegate?**

Middleware components are chained. Each receives an HttpContext and a RequestDelegate (next). Calling next() passes control to the next component. Not calling it short-circuits the pipeline. GlobalExceptionMiddleware wraps next() in try/catch.

> **REAL-WORLD** Like an assembly line where each worker can either pass the item on (next) or stop the belt.

Q33 **What is a Global Exception Handler? How did LifeTrack implement it?**

GlobalExceptionMiddleware catches unhandled exceptions from the entire pipeline. It maps DomainException → 400, NotFoundException → 404, others → 500. Registered via app.UseGlobalExceptionHandler() as the first middleware.

> **REAL-WORLD** Like a building's central fire-alarm: any unhandled smoke anywhere triggers one consistent response.

Q34 **What is an Action Filter? Describe DomainExceptionFilter in LifeTrack.**

DomainExceptionFilter implements IExceptionFilter. On DomainException it sets context.Result = ObjectResult(400). On NotFoundException it sets 404. Registered globally so every controller action gets this behaviour.

> **REAL-WORLD** Like a translator at a help desk who converts 'business problem' into the right official rejection slip.

Q35 How did LifeTrack implement JWT authentication without UseAuthentication()?

JwtAuthFilter is a global IAsyncAuthorizationFilter registered via o.Filters.Add<JwtAuthFilter>(). It reads the Bearer token from the Authorization header, validates it with JwtSecurityTokenHandler, and sets context.HttpContext.User.

REAL-WORLD Like a bouncer at every door checking your wristband, instead of one bouncer at the entrance.

Q36 What is JWT? What are its three parts?

JSON Web Token for stateless authentication. Header (algorithm), Payload (claims: sub, email, role, exp), Signature (HMAC-SHA256). LifeTrack JwtTokenService generates tokens with user claims; Angular AuthService decodes them client-side.

REAL-WORLD Like a tamper-proof festival wristband: header (type), payload (your name/zone), signature (hologram) proving it's real.

Q37 What claims does LifeTrack embed in the JWT?

sub (UserID), email, jti (unique token ID), NameIdentifier (UserID), Name, Email, Role (e.g. "ClinicalTrialManager"). Angular reads role/sub claims to determine dashboard routing and RoleGuard checks.

REAL-WORLD Like the info printed on that wristband: your ID, your access zone (role), and its expiry time.

Q38 What is the [AllowAnonymous] attribute? Which LifeTrack endpoints use it?

Bypasses the JwtAuthFilter for that endpoint. POST /api/auth/login and POST /api/auth/register/patient are AllowAnonymous — they must be reachable without a token.

REAL-WORLD Like a 'staff only' sign removed from the front door so anyone can walk into the lobby (login page).

Q39 What is the RoleAuthorizeFilter in LifeTrack? How does it work?

Custom filter that reads ClaimTypes.Role from the authenticated principal and checks against the allowed roles passed to [RoleAuthorize(RolesEnum.Admin, ...)]. Returns 403 Forbidden if role not permitted.

REAL-WORLD Like a club checking not just that you have a wristband, but that it's a VIP one for the VIP lounge.

Q40 What is the difference between Authentication and Authorization?

Authentication = proving identity (JWT validates who you are). Authorization = proving permission (RoleAuthorizeFilter checks what you are allowed to do). LifeTrack does auth first (JwtAuthFilter), then role check (RoleAuthorize attribute).

REAL-WORLD Authentication is proving who you are at the door; authorization is whether your badge opens the server room.

Q41 What is CORS? Why is it needed in LifeTrack?

Cross-Origin Resource Sharing — browsers block requests to different origins. Angular (localhost:4200) calls LifeTrack APIs (localhost:5033 etc.). CORS policy "AngularDevClient" whitelists the Angular origins, headers, and methods.

REAL-WORLD Like a building intercom that only buzzes in visitors from a pre-approved guest list, not any stranger.

Q42 Why does LifeTrack use WithOrigins() instead of AllowAnyOrigin()?

AllowAnyOrigin() combined with AllowCredentials() is invalid and broadens the attack surface. LifeTrack uses an explicit allow-list (localhost:4200, 62536) for security. Only the Gateway uses AllowCredentials().

> **REAL-WORLD** Like a guest list naming four specific friends rather than 'open bar, anyone welcome' — a known allow-list is far safer than opening the API to every website.

Q43 What is Swagger/OpenAPI? How did LifeTrack configure it?

Swagger generates interactive API documentation from code. LifeTrack uses Swashbuckle (AddSwaggerGen) with a Bearer security definition so testers can paste JWT tokens and call protected endpoints from the Swagger UI.

> **REAL-WORLD** Like a glass display case showing testers exactly which buttons the API has and letting them press each.

Q44 What is IMemoryCache? How is it used in LifeTrack?

In-process key-value cache. LifeTrack services (AuthService, DocumentService) cache expensive DB list queries under string keys. On mutations, InvalidateListCache() calls _cache.Remove(key) to force refresh on next request.

> **REAL-WORLD** Like keeping a frequently-checked phone number on a sticky note instead of looking it up in the directory each time.

Q45 Why did LifeTrack face issues with IMemoryCache wildcard key removal?

IMemoryCache.Remove() requires the exact key — there are no wildcard patterns. When multiple keys were in use (page 1, page 2, different filters), each had to be tracked explicitly and removed one by one.

> **REAL-WORLD** Like trying to erase 'all notes starting with M' from a whiteboard that has no such bulk-erase button — you must wipe each by name.

Q46 What is BCrypt? Why is it used for password hashing?

BCrypt is an adaptive hashing algorithm with a built-in work factor (cost). It automatically salts passwords, making rainbow table attacks impractical. LifeTrack uses BCryptPasswordHasher wrapping BCrypt.Net-Next.

> **REAL-WORLD** Like a paper shredder for passwords: even with the shredded bits, no one can reassemble the original.

Q47 What is the difference between AddSingleton, AddScoped, and AddTransient?

Singleton: one instance for app lifetime. Scoped: one per HTTP request. Transient: new instance every injection. LifeTrack uses AddScoped for DbContext, repositories, and services — one per request is the correct lifetime.

> **REAL-WORLD** Singleton = one office printer for all; Scoped = a fresh notepad per visitor; Transient = a new pen every time you write.

Q48 What is IHttpContextAccessor? How is it used in LifeTrack?

Provides access to the current HttpContext outside of controllers. LifeTrack DbContexts use it to extract the user ID from the JWT and stamp audit rows with ChangedByUserID automatically on SaveChanges.

> **REAL-WORLD** Like a receptionist who can always see which visitor is currently at the desk, from anywhere in the building.

Q49 What is the difference between UseAuthentication() and UseAuthorization()?

UseAuthentication() populates HttpContext.User from the token. UseAuthorization() enforces [Authorize] policies. LifeTrack bypasses both middleware-level calls by handling everything in JwtAuthFilter.

REAL-WORLD Like two old switches you no longer flip because the new smart panel (filter) already handles the lights.

Q50 What is an ApiResponse<T>? How does it standardize responses?

A wrapper DTO from Shared.CL with Success, Data<T>, Message properties. Used in error paths (ApiResponse<object>.Fail("message")). Ensures callers always receive a consistent JSON structure.

REAL-WORLD Like a standard company letterhead so every reply — good news or bad — looks the same.

Q51 What is Postman? How would you test the LifeTrack login endpoint?

HTTP client for manual API testing. POST to http://localhost:5033/api/auth/login with JSON body {"email":"admin@lifetrack.com","password":"Admin@123"}. Capture the token from the response and set it as Bearer in subsequent requests.

REAL-WORLD Like a free coffee voucher app — POST to /login, get a stamp (token), show it for every refill.

Q52 What is the OpenAPI specification? What does it describe?

A language-agnostic standard for describing RESTful APIs. Describes endpoints, parameters, request/response schemas, authentication schemes. Swagger UI renders it. LifeTrack generates it via Swashbuckle at /swagger/v1/swagger.json.

REAL-WORLD Like the architectural drawing standard that lets any builder read any blueprint.

Q53 What is integration testing? How would you write one for AuthController?

Tests the full HTTP pipeline against a real (or in-memory) database. Use WebApplicationFactory<Program> + EF Core in-memory provider. POST /api/auth/login and assert 200 + token present. LifeTrack uses MSTest + WebApplicationFactory.

REAL-WORLD Like a dress rehearsal on the real stage with stand-in actors (in-memory DB) before opening night.

Q54 What is the difference between unit testing and integration testing?

Unit tests isolate one class (mock dependencies). Integration tests test the full stack end-to-end. LifeTrack's identity service integration tests test the real auth pipeline with an in-memory DB.

REAL-WORLD Unit test = checking one Lego brick; integration test = checking the whole assembled wall holds.

Q55 What is xUnit vs NUnit vs MSTest?

All are .NET test frameworks. xUnit uses [Fact]/[Theory], NUnit uses [Test]/[TestCase], MSTest uses [TestMethod]/[DataTestMethod]. LifeTrack uses MSTest for integration tests.

REAL-WORLD Three brands of the same measuring tape — all measure, just different markings ([Fact]/[Test]/[TestMethod]).

Q56 What is Moq? How would you mock IAdverseEventService?

A mocking library. var mock = new Mock<IAdverseEventService>(); mock.Setup(s => s.GetAsync(1)).ReturnsAsync(fakeDto); Then inject mock.Object into the controller under test.

REAL-WORLD Like a crash-test dummy standing in for a real passenger so you can test the car safely.

Q57 What is the ValidateModelFilter in LifeTrack?

An IActionFilter that checks ModelState.IsValid before the action runs. Returns 422 UnprocessableEntity with validation errors if invalid. This removes the repetitive if (!ModelState.IsValid) check from every action.

REAL-WORLD Like a quality inspector who rejects faulty forms (422) before they reach the production line.

Q58 What is the ActivityLogFilter in LifeTrack?

An IResultFilter (runs after action completes). Logs each successful request with the authenticated user ID, HTTP method, path, and timestamp. Used for observability and audit trails across all services.

REAL-WORLD Like a security camera quietly logging who entered and when, after each successful entry.

Q59 What is the difference between 400 Bad Request and 422 Unprocessable Entity?

400 is a general client error (malformed request, business rule violation). 422 specifically means the request was syntactically valid but semantically invalid (validation errors on the model). LifeTrack uses 422 for ModelState errors.

REAL-WORLD 400 is 'your request makes no sense'; 422 is 'I understood it, but the age can't be -5'.

Q60 How would you version an API in ASP.NET Core?

Via URL segments (/api/v1/adverse-events), query strings (?api-version=1.0), or headers. Use the Asp.Versioning.Mvc NuGet package. LifeTrack currently has a single version ("v1") but Swagger is already named "v1" in anticipation.

REAL-WORLD Like book editions (v1, v2) sitting side by side on the shelf so old readers aren't broken.

Q61 What is WCF? When would you use SOAP over REST?

Windows Communication Foundation — for SOAP services. SOAP is preferred when formal contracts (WSDL), WS-Security, ACID transactions, or strong typing across enterprise systems are needed. LifeTrack uses REST; SOAP would apply in legacy hospital system integrations.

REAL-WORLD WCF/SOAP is the formal notarized contract you'd use with a bank's legacy mainframe; REST is the casual app.

Q62 What is IHttpClientFactory? How does LifeTrack use it?

Manages pooled HttpClient instances to avoid socket exhaustion. LifeTrack registers "PatientApi" named client in Authentication.API/Program.cs. AuthController uses it to call Patient.API during patient self-registration.

REAL-WORLD Like a taxi dispatch office that reuses a fleet of cars instead of buying a new car per trip.

Q63 What is the difference between synchronous and asynchronous controllers?

Async controllers use async/await and Task-returning actions, freeing the thread pool thread while I/O is pending. LifeTrack uses async Task<ActionResult<T>> throughout for better scalability.

> **REAL-WORLD** Async = the waiter takes other tables' orders while your food cooks; sync = he stands frozen at your table.

Q64 What is appsettings.json vs appsettings.Development.json?

appsettings.json is base config; appsettings.Development.json overrides it in Development environment. LifeTrack keeps SQL connection strings and JWT secrets in appsettings.json, overridden per environment.

> **REAL-WORLD** Like a master settings sheet plus a sticky note that overrides a couple of values only while testing.

Q65 What is the IConfiguration interface?

Provides access to app configuration from multiple sources (JSON, env vars, command line). JwtAuthFilter reads _config["Jwt:Secret"] to get the signing key. Production overrides via JWT_SECRET environment variable.

> **REAL-WORLD** Like the universal remote that reads settings from whichever source is plugged in.

Q66 What is dependency injection in ASP.NET Core?

Built-in IoC container resolves service dependencies automatically. LifeTrack registers IAdverseEventRepository → AdverseEventRepository (AddScoped). Controllers/services receive resolved instances via constructor.

> **REAL-WORLD** Like a kitchen that's handed its ingredients rather than going to buy them — easier to swap suppliers.

Q67 What is a DomainException vs NotFoundException in LifeTrack?

Both extend Exception and live in Shared.CL.Exceptions. DomainException represents a business rule violation (400). NotFoundException represents a missing entity (404). They are caught by GlobalExceptionMiddleware and DomainExceptionFilter.

> **REAL-WORLD** Like two rejection stamps: 'Against policy' (400) vs 'File not found' (404).

Q68 Explain the Saga pattern used in RegisterPatient.

A distributed transaction pattern. Step 1: create User in Auth.API. Step 2: auto-login for token. Step 3: POST patient record to Patient.API. If any step fails, the compensating action TryDeleteUserAsync() rolls back the User creation.

> **REAL-WORLD** Booking a flight + hotel + car: if the car fails, you cancel the hotel and flight to stay consistent.

Q69 How does LifeTrack handle pagination consistently across services?

Every service has its own PagedResult<T> DTO with Items, TotalCount, Page, PageSize. Repositories use .CountAsync() for total and .Skip((page-1)*pageSize).Take(pageSize) for data. Angular uses these values to render paginators.

> **REAL-WORLD** Like every shop branch using the same '20 per page, here's the total' receipt format.

Q70 What is JsonStringEnumConverter and why is it useful?

Serializes/deserializes C# enums as their string names instead of integer values. LifeTrack uses it so the JSON payload shows severity: "Mild" instead of severity: 1, making it self-documenting for consumers.

REAL-WORLD Like printing 'Severe' on a label instead of a cryptic code '1' that no customer understands.

Q71 What is the difference between AddControllers and AddControllersWithViews?

AddControllers registers only API controllers (no view support). AddControllersWithViews adds Razor view rendering. LifeTrack uses AddControllers since it is a pure API project.

REAL-WORLD Like a delivery-only kitchen vs one that also seats diners — you pick the lean option for an API.

Q72 What is Content Negotiation in Web APIs?

The client specifies acceptable response formats via the Accept header (e.g., application/json). ASP.NET Core selects the appropriate formatter. LifeTrack returns JSON exclusively.

REAL-WORLD Like a restaurant asking 'table or takeaway?' — the client says which format it wants back.

Q73 What are HTTP PUT vs PATCH?

PUT replaces the entire resource. PATCH partially updates it. LifeTrack's AdverseEventsController uses PUT for full updates. PATCH (with JsonPatch) is not implemented but the Gateway routes support PATCH method.

REAL-WORLD PUT repaints the whole wall; PATCH just touches up one scratch.

Q74 What is the difference between StatusCode(201, value) and Created(uri, value)?

Created() adds a Location header with the resource URI (REST best practice). StatusCode(201, value) just sets the status and body. LifeTrack uses StatusCode(201, new {eventID}) for simplicity.

REAL-WORLD Created() leaves a forwarding address (Location header); StatusCode(201) just says 'done'.

Q75 What is NoContent() and when should it be returned?

204 No Content — the request succeeded but there is nothing to return. LifeTrack returns NoContent() from PUT endpoints because the client already has the updated data it just sent.

REAL-WORLD Like a silent nod after you hand over a form — 'received, nothing to hand back' (204).

Q76 What are the benefits of using interfaces for repositories and services?

Loose coupling, dependency injection, testability (mock the interface), and interchangeability. LifeTrack has IAdverseEventRepository, IAdverseEventService etc., allowing the controller to be tested with fake implementations.

REAL-WORLD Like job descriptions: you hire 'a driver' (interface), not one specific person, so you can swap drivers freely.

Q77 What is Ocelot? What role does it play in LifeTrack?

Ocelot is a .NET API Gateway library. In LifeTrack, Gateway.API (port 5046) uses Ocelot to route requests from Angular to the correct downstream service based on the URL prefix (e.g., /gateway/adverse-events → localhost:5159).

REAL-WORLD Like a hotel concierge who routes your request to the right department — you never see the back offices.

Q78 What is the GlobalConfiguration.BaseUrl in ocelot.json?

Declares the external-facing URL of the gateway itself (http://localhost:5046). Ocelot uses this when building redirect URLs or internal references. Angular's environment.apiUrl points to this gateway.

REAL-WORLD Like the building's official street address printed on the directory board.

Q79 What is AddMemoryCache()? What are its limitations?

Registers IMemoryCache in DI — an in-process thread-safe cache. Limitations: data lost on restart, not shared across multiple service instances, and wildcard key removal is not supported (LifeTrack had to maintain explicit key lists).

REAL-WORLD Like a memo board on one desk — handy, but the next desk can't see it and it's wiped on restart.

Q80 How would you implement rate limiting in LifeTrack?

In .NET 8, use the built-in RateLimiter middleware: app.UseRateLimiter(). Configure sliding window or fixed window policies. Add to the Gateway so it protects all downstream services from abuse.

REAL-WORLD Like a 'one coffee per minute' rule at a free sample stand to stop one person grabbing the whole tray.

Q81 What is the difference between ILogger and ILogger<T>?

ILogger<T> is the generic version that adds the type name as the log category. LifeTrack's AuthController injects ILogger<AuthController> — log entries show "Authentication.API.Controllers.AuthController" as the source.

REAL-WORLD Like a labelled logbook ('AuthController log') vs an unlabelled generic notebook.

Q82 What is problem details format (RFC 7807)?

A standardized JSON error format with type, title, status, detail, instance fields. ASP.NET Core's built-in validation errors follow this. LifeTrack's custom ErrorDetail DTO has a similar structure.

REAL-WORLD Like a standard insurance-claim rejection letter format the whole industry recognizes.

Q83 What is ClockSkew in JWT validation? What value does LifeTrack use?

Tolerates small time differences between token issuer and validator clocks. JwtAuthFilter sets ClockSkew = TimeSpan.FromSeconds(30) — a token expiring at T is still accepted until T+30 seconds.

REAL-WORLD Like giving a 30-second grace period on a parking meter so a slightly-fast clock doesn't fine you.

Q84 What is the purpose of the Jti claim in JWTs?

JWT ID — a unique identifier (Guid.NewGuid()) for each token. Enables token revocation via a blocklist (check if jti is in a revoked set). LifeTrack generates one per token but does not currently implement a revocation store.

REAL-WORLD Like a unique ticket number on each cinema stub so a specific ticket can be cancelled later.

Q85 How would you implement refresh tokens in LifeTrack?

Issue a long-lived refresh token alongside the short-lived JWT. Store it in the DB. POST /api/auth/refresh endpoint: validate the refresh token, issue a new JWT. Angular would call this before the JWT expires.

REAL-WORLD Like a hotel key-card that re-issues itself before checkout so you're never locked out mid-stay.

Q86 What is the ValidateModelFilter advantage over [ValidateModel] in each action?

Registered globally in Program.cs via o.Filters.Add<ValidateModelFilter>(), it applies to ALL actions automatically. Eliminates repetitive if (!ModelState.IsValid) return ValidationProblem() checks. LifeTrack uses this shared filter across all 8 services.

REAL-WORLD Like a standing rule the whole building follows, so no single office forgets the safety check.

Q87 What is ActionResult vs IActionResult vs ObjectResult?

IActionResult is the base interface. ActionResult<T> is generic (type-safe, Swagger-friendly). ObjectResult is a concrete result that sets status code and value. context.Result = new ObjectResult(payload){StatusCode=401} is used directly in JwtAuthFilter.

REAL-WORLD Three labels for the same parcel: vague 'item', typed 'fragile glass' (T), and the raw box (ObjectResult).

Q88 What is app.UseGlobalExceptionHandler()? Where is it defined?

An extension method in Shared.CL/Extensions/ApplicationBuilderExtensions.cs that calls app.UseMiddleware<GlobalExceptionMiddleware>(). It's registered first in every service's Program.cs to ensure it wraps the entire pipeline.

REAL-WORLD Like the master fire-alarm wiring kept in the building's shared electrical room (Shared.CL).

Q89 How does LifeTrack prevent CORS errors for preflight (OPTIONS) requests?

CORS policy explicitly whitelists OPTIONS in WithMethods(). Browser sends OPTIONS preflight before PUT/DELETE/custom-header requests. Without OPTIONS in the allowed methods, Angular calls would fail before the real request.

REAL-WORLD Like the airport letting the 'are you cleared to fly?' pre-check (OPTIONS) pass before the real boarding.

Q90 What is the difference between async Task<IActionResult> and async Task<ActionResult<T>>?

Task<IActionResult> is untyped — Swagger cannot infer the response schema. Task<ActionResult<T>> allows Swagger to show the typed response. LifeTrack uses ActionResult<PagedResult<T>> for typed endpoints and ActionResult for mutations.

REAL-WORLD Like a parcel labelled with exact contents (T) so the courier app shows a picture, vs a plain box.

Q91 What is appsettings.json hierarchy? Which overrides which?

Base: appsettings.json → Environment-specific: appsettings.Development.json (overrides base) → Environment variables (override JSON) → Command-line args (highest priority). LifeTrack's JwtAuthFilter prefers JWT_SECRET environment variable over appsettings.json.

> **REAL-WORLD** Like a chain of overrides: company policy < branch policy < today's manager note < a direct phone order.

Q92 What is EF Core's IQueryable vs IEnumerable?

IQueryable<T> builds a SQL query incrementally — .Where() adds a SQL WHERE clause. IEnumerable<T> executes in-memory after data is fetched. LifeTrack repositories return IQueryable to allow chaining filters before materializing with .ToListAsync().

> **REAL-WORLD** IQueryable is a shopping list you keep adding to before checkout; IEnumerable is the bag after you've paid.

Q93 What is optimistic concurrency in EF Core?

A [Timestamp] or [ConcurrencyCheck] column detects if a row changed between read and update. If so, SaveChanges throws DbUpdateConcurrencyException. Prevents lost updates when two users edit the same record. LifeTrack documents could benefit from this.

> **REAL-WORLD** Like two people editing one shared doc — a version stamp warns 'someone changed this since you opened it'.

Q94 What are the 6 roles in LifeTrack and how are they stored?

Admin (1), ClinicalTrialManager (2), Investigator (3), Patient (4), RegulatoryOfficer (5), DataManager (6). Stored as UserRole enum and as RolesEnum in Shared.CL. The RoleID FK in the User table references the Role table seeded by DbSeeder.

> **REAL-WORLD** Like six staff badges (Admin, CTM, Investigator, Patient, Regulatory, DataManager) each opening different doors.

Q95 What is EF Core's ConfigureWarnings? How did LifeTrack use it?

Suppresses specific EF Core warnings. DocumentCompliance.API Program.cs: .ConfigureWarnings(w => w.Ignore(RelationalEventId.PendingModelChangesWarning)) — suppresses the warning about pending model changes after adding migrations during active development.

> **REAL-WORLD** Like muting one specific over-sensitive smoke detector that keeps beeping during cooking.

Q96 EF Core's EnsureCreatedAsync creates the schema if it doesn't exist but does NOT use migrations. Migrate() applies pending migrations. LifeTrack uses EnsureCreatedAsync for speed in dev but recommends dotnet ef database update for production.

See above — this is a full repeat entry combined with the next new one.

> **REAL-WORLD** EnsureCreated builds the house once; Migrate is the renovation crew that applies each change order.

Q97 What is EF Core's EnsureCreatedAsync()? How is it different from Migrate()?

EnsureCreatedAsync creates the schema if it doesn't exist but does NOT use migrations. Migrate() applies pending migrations. LifeTrack uses EnsureCreatedAsync for speed in dev but recommends dotnet ef database update for production.

> **REAL-WORLD** EnsureCreated builds the house from scratch; Migrate applies tracked renovation plans one by one.

Q98 What is DbSet<T> in EF Core?

Represents a table and exposes LINQ query methods (Where, Include, FirstOrDefaultAsync, etc.) and mutation methods (Add, Remove). AdverseEventDbContext has DbSet<AdverseEventRecord> and DbSet<Deviation>.

> **REAL-WORLD** Like one labelled drawer in the filing cabinet that holds all the 'adverse event' folders.

Q99 What is a DTO (Data Transfer Object)? Why separate from the model?

DTOs define what goes in/out of the API, decoupled from the database model. Prevents over-posting attacks (exposing DB fields to clients) and allows versioning. LifeTrack has separate CreateAdverseEventRequest, UpdateAdverseEventRequest, AdverseEventResponse DTOs.

> **REAL-WORLD** Like a customs declaration form — only the fields you're meant to show, not your whole suitcase.

Q100 What is over-posting/mass assignment attack? How does LifeTrack prevent it?

A client sends extra fields in the request body that get bound to model properties they shouldn't control. DTOs prevent this — only explicitly declared DTO properties are bound, not the full entity.

> **REAL-WORLD** Like a form that ignores extra scribbles in the margin so a prankster can't sneak 'role=admin' onto it.

Q101 What is a named HTTP client in ASP.NET Core?

A pre-configured HttpClient instance registered with a name. AddHttpClient("PatientApi", client => client.BaseAddress = ...) in Authentication.API. Injected via IHttpClientFactory.CreateClient("PatientApi").

> **REAL-WORLD** Like the taxi office's pre-arranged car for one specific regular route (the 'PatientApi' cab).

Part II Microservices Architecture

Decomposition, gateways, inter-service communication, sagas, security and deployment. · **100 questions**, each with an answer, project code where relevant, and a real-world analogy.

Q1 What is microservices architecture? How is LifeTrack structured as microservices?

An architecture style where an application is split into small, independently deployable services each owning its domain. LifeTrack has 8 microservices: Gateway.API, Authentication.API, ProtocolSite.API, Patient.API, AdverseEvent.API, DocumentCompliance.API, AnalyticsReport.API, Notification.API.

REAL-WORLD Like a food court: separate stalls (services) for pizza, sushi, coffee — each runs alone but shares the mall.

Q2 What are the advantages of microservices over a monolith?

Independent deployment, technology diversity per service, fault isolation, independent scaling, smaller codebases, and team autonomy. LifeTrack allows Hari's AdverseEvent.API to be deployed independently from Naveen's Patient.API.

REAL-WORLD Like specialist shops vs one giant department store — you can renovate the bakery without closing the whole mall.

Q3 What are the challenges of microservices?

Distributed system complexity (network failures, latency), data consistency across services, service discovery, distributed tracing, testing complexity, and operational overhead. LifeTrack addresses some via the Saga pattern and shared Shared.CL library.

REAL-WORLD Like coordinating 8 separate shops instead of one — more phone calls, deliveries and things that can go wrong.

Q4 What is the difference between microservices and monolithic architecture?

Monolith: single deployable unit, single database, simpler but hard to scale parts independently. Microservices: many deployable units, database per service, complex but independently scalable. LifeTrack started as a monolith (Razor Pages) and migrated to microservices.

REAL-WORLD A monolith is one big restaurant; microservices are a food court of independent kitchens.

Q5 What is an API Gateway? What does LifeTrack's Gateway.API do?

A single entry point that routes requests to backend services. Gateway.API (port 5046, Ocelot) routes Angular requests: /gateway/adverse-events → AdverseEvent.API:5159, /gateway/patients → Patient.API:5095, etc.

REAL-WORLD Like a hotel front desk that directs you to the right department — you only ever talk to reception.

Q6 What are the 8 LifeTrack microservices and their ports?

Gateway.API:5046, Authentication.API:5033, ProtocolSite.API:5193, Patient.API:5095, AdverseEvent.API:5159, DocumentCompliance.API:5232, AnalyticsReport.API:5062, Notification.API:5210.

REAL-WORLD Like a directory board listing 8 offices and their floor numbers (ports).

Q7 What is the database-per-service pattern? How does LifeTrack implement it?

Each service owns its own database to ensure loose coupling. LifeTrack uses two main databases: governanceDb (Authentication, DocumentCompliance) and clinicalDb (ProtocolSite, Patient, AdverseEvent, AnalyticsReport). Notification has its own DB.

> **REAL-WORLD** Like each shop keeping its own till and stockroom rather than sharing one cash register.

Q8 What is service discovery? How does LifeTrack handle it?

Finding the location of a service dynamically. LifeTrack uses static discovery — hardcoded ports and hosts in ocelot.json (localhost:5159, etc.) and appsettings.json ServiceUrls. Production would use Consul or Kubernetes DNS.

> **REAL-WORLD** Like asking reception 'which floor is Dr. Patel on today?' instead of memorizing it.

Q9 What is inter-service communication? How does LifeTrack implement it?

Communication between microservices. LifeTrack uses synchronous HTTP (IHttpClientFactory + HttpClient) for the Patient registration saga in AuthController. There is no message queue (async messaging) in the current version.

> **REAL-WORLD** Like shop staff phoning each other to complete a customer's combined order.

Q10 What is the difference between synchronous and asynchronous inter-service communication?

Synchronous (HTTP/gRPC): caller waits for response, simpler but creates temporal coupling. Async (message queues: RabbitMQ, Azure Service Bus): fire-and-forget, decoupled, resilient. LifeTrack uses sync HTTP; async messaging would be the production improvement.

> **REAL-WORLD** Sync = waiting on the phone for the other shop to answer; async = leaving a voicemail and moving on.

Q11 What is gRPC? How does it compare to REST for microservice communication?

Google RPC using Protocol Buffers — faster, strongly typed, bidirectional streaming, but less human-readable. REST (JSON) is simpler and more debuggable. LifeTrack uses REST internally; gRPC would be optimal for high-frequency internal service calls.

> **REAL-WORLD** gRPC is a private high-speed intercom between staff; REST is the public phone line anyone understands.

Q12 What is the Shared.CL class library in LifeTrack?

A shared .NET class library referenced by all microservices. Contains: JwtAuthFilter, DomainExceptionFilter, ActivityLogFilter, ValidateModelFilter, GlobalExceptionMiddleware, shared DTOs (ErrorDetail, ApiResponse), exceptions (DomainException, NotFoundException), and RolesEnum.

> **REAL-WORLD** Like the shared staff handbook every shop in the mall follows (security, complaint forms).

Q13 What are the problems with shared libraries in microservices?

Shared libraries create coupling — a change in Shared.CL requires recompiling and redeploying all services. Versioning is tricky. LifeTrack accepts this trade-off for cross-cutting concerns (auth, exception handling) that are stable.

> **REAL-WORLD** Like a shared handbook — handy, but reprinting it forces every shop to re-read and re-train.

Q14 **What is the Circuit Breaker pattern? Is it used in LifeTrack?**

Stops calling a failing downstream service after N failures, "opening" the circuit. Requests fail fast without waiting for timeout. Not yet in LifeTrack — the Patient registration saga uses simple try/catch with rollback instead of a circuit breaker.

REAL-WORLD Like a fuse that trips after repeated faults so a broken appliance doesn't burn down the house.

Q15 **What is the Bulkhead pattern?**

Isolates components so a failure in one does not cascade. In LifeTrack, each microservice is a separate process — AdverseEvent.API crashing does not bring down Authentication.API (bulkhead by process isolation).

REAL-WORLD Like watertight compartments on a ship — one flooded section doesn't sink the whole vessel.

Q16 **What is JWT authentication in a microservices context? How does LifeTrack handle it?**

Each service independently validates the JWT using the shared secret without calling the auth service. JwtAuthFilter in Shared.CL is registered in every service's Program.cs, so any service can validate tokens autonomously.

REAL-WORLD Like each shop checking your festival wristband itself, instead of phoning the main gate every time.

Q17 **What is the difference between centralized and distributed authentication in microservices?**

Centralized: all requests route through auth service (single point of failure). Distributed: each service validates the JWT independently (LifeTrack's approach) — more resilient, no inter-service auth dependency.

REAL-WORLD Centralized = one guard everyone queues at; distributed = every door checks your badge independently.

Q18 **What is the role of the ClaimsPrincipal in LifeTrack microservices?**

After JwtAuthFilter validates the token, it sets context.HttpContext.User = principal. The principal carries claims (user ID, role) that RoleAuthorizeFilter and controller actions read to make authorization decisions.

REAL-WORLD Like a verified visitor badge each office can read to know your name and clearance.

Q19 **What is eventual consistency? Is it relevant to LifeTrack?**

In distributed systems, data across services will eventually be consistent but not immediately. LifeTrack's patient registration saga is an example — if Patient.API fails, Auth.API compensates by deleting the user, eventually restoring consistency.

REAL-WORLD Like a bank transfer that shows in your app a moment later — it WILL match, just not instantly.

Q20 **What is a compensating transaction? Give a LifeTrack example.**

An action that undoes a completed step when a later step fails. In RegisterPatient, if Patient.API returns non-2xx, TryDeleteUserAsync() deletes the just-created user account — compensating the first step.

REAL-WORLD Like cancelling a hotel booking after the connecting flight falls through, to undo a half-done trip.

Q21 What is the two-phase commit problem in microservices?

2PC requires all participants to agree before committing, causing coupling and blocking. Microservices prefer the Saga pattern instead. LifeTrack's registration saga is a simplified choreography saga without a two-phase lock.

REAL-WORLD Like needing two people to sign at the exact same instant — impractical, so you avoid it.

Q22 What is the Outbox Pattern?

Reliably publish events by writing to an "outbox" table in the same DB transaction as the business data, then asynchronously publishing to the message broker. LifeTrack would benefit from this to make the patient-creation event reliable.

REAL-WORLD Like dropping your outgoing letters in your own mailbox so they're guaranteed to be posted later.

Q23 What is a Message Queue? How would it improve LifeTrack?

Decoupled async communication (e.g., RabbitMQ, Azure Service Bus). Instead of synchronous HTTP to Patient.API, Auth.API publishes a "UserRegistered" event. Patient.API subscribes and creates the patient record asynchronously — more resilient.

REAL-WORLD Like a restaurant order queue — kitchen pulls tickets when ready, so a rush doesn't lose orders.

Q24 What is the difference between orchestration and choreography in sagas?

Orchestration: a central coordinator (orchestrator) tells services what to do (easier to debug). Choreography: each service reacts to events autonomously (more decoupled). LifeTrack's registration is orchestration — AuthController coordinates steps.

REAL-WORLD Orchestration = a conductor cueing each musician; choreography = dancers each reacting to the music.

Q25 What is health check endpoint? Why is it important in microservices?

GET /health returns the service's operational status (healthy/degraded/unhealthy). Used by load balancers and orchestrators (Kubernetes) to route traffic only to healthy instances. LifeTrack services should add app.MapHealthChecks("/health") with EF Core health checks.

REAL-WORLD Like a shop's 'Open/Closed' light so the mall directory only sends customers to open shops.

Q26 What is distributed tracing? How would you add it to LifeTrack?

Tracking a request as it flows through multiple services (correlation ID). Tools: OpenTelemetry + Jaeger/Zipkin. Add a TraceId header in Gateway.API and forward it downstream. The GlobalExceptionMiddleware already captures TraceIdentifier.

REAL-WORLD Like a parcel's full journey log across every depot, tied together by one tracking number.

Q27 What is Serilog? How would you use it in LifeTrack?

A structured logging library for .NET. .UseSerilog() in Program.cs. Outputs structured JSON logs to console, files, Seq, or Elasticsearch. ActivityLogFilter already logs per request — Serilog would replace the default ILogger with structured sinks.

REAL-WORLD Like a structured CCTV system that timestamps and labels every event for easy review.

Q28 What is the difference between horizontal and vertical scaling?

Horizontal: add more instances of a service (scale out). Vertical: add more CPU/RAM to existing instance (scale up). Microservices favor horizontal scaling. LifeTrack could run 3 instances of AdverseEvent.API behind a load balancer.

REAL-WORLD Horizontal = hiring more cashiers; vertical = giving one cashier a faster till.

Q29 What is Docker? How would you containerize LifeTrack?

Docker packages an app and its dependencies into a container image. Create a Dockerfile per service: base ASP.NET Core image, copy published output, ENTRYPOINT ["dotnet", "Authentication.API.dll"]. docker-compose.yml orchestrates all 8 services + SQL Server.

REAL-WORLD Like shipping a meal in a sealed lunchbox that works in any kitchen, fridge or microwave.

Q30 What is Kubernetes? What would it offer LifeTrack?

Container orchestration platform. For LifeTrack: auto-scaling pods, self-healing (restart crashed services), service discovery via DNS, rolling deployments, config maps for appsettings. Each LifeTrack service would be a Kubernetes Deployment.

REAL-WORLD Like a smart building manager that restarts failed lifts, adds lifts at rush hour and routes traffic.

Q31 What is the difference between a rolling update and a blue-green deployment?

Rolling: gradually replace old pods with new ones. Blue-green: run old (blue) and new (green) simultaneously, switch traffic at once. Blue-green has zero downtime and instant rollback. LifeTrack in production would benefit from blue-green for the Gateway.

REAL-WORLD Rolling = repainting rooms one at a time; blue-green = building an identical house and moving over instantly.

Q32 What is a CI/CD pipeline? What would one look like for LifeTrack?

CI: build, run tests, lint on every commit. CD: deploy automatically to staging/production. GitHub Actions workflow: checkout → dotnet build → dotnet test → docker build → push to registry → kubectl apply. One pipeline per microservice.

REAL-WORLD Like an automated factory line: commit code, it's tested, packaged and shipped without manual steps.

Q33 What is GitHub Actions? How would you create a workflow for Authentication.API?

on: [push] → jobs: build (ubuntu-latest) → steps: dotnet restore, dotnet build, dotnet test, docker build/push. Secrets (SQL connection string, JWT secret) stored in GitHub Secrets and injected as environment variables.

REAL-WORLD Like a robot assistant that runs your checklist (build, test, deploy) every time you change a recipe.

Q34 What is the gateway aggregation pattern?

The gateway aggregates calls to multiple downstream services into a single response. LifeTrack's Gateway.API currently only proxies (no aggregation). Admin dashboard could benefit from an aggregation endpoint combining user + protocol + adverse event counts.

REAL-WORLD Like a concierge who fetches your dry-cleaning AND coffee in one trip instead of you visiting both.

Q35 What is service mesh? Give an example (Istio, Linkerd).

Infrastructure layer managing service-to-service communication: mTLS, load balancing, retries, circuit breaking, observability. Istio or Linkerd run as sidecar proxies alongside each service. LifeTrack would use a service mesh in Kubernetes for automatic mTLS.

REAL-WORLD Like a city's traffic-control grid managing every road between buildings (retries, tolls, cameras).

Q36 What is an anti-corruption layer in microservices?

A translation layer between two services with different domain models, preventing one service's model from corrupting another's. If LifeTrack integrated with an external hospital system's SOAP API, an anti-corruption layer would translate their patient model to LifeTrack's.

REAL-WORLD Like a translator between two companies that use different terms for the same thing, keeping each pure.

Q37 What is the strangler fig pattern?

Incrementally replace a monolith by routing specific endpoints to new microservices while the monolith handles the rest. LifeTrack itself was migrated this way — starting as a Razor Pages monolith, then extracting service by service.

REAL-WORLD Like renovating an old house room by room while still living in it, until it's fully modern.

Q38 What is the CQRS pattern?

Command Query Responsibility Segregation — separate read (query) models from write (command) models. LifeTrack's AnalyticsReport.API could benefit: queries read a denormalized read model, commands update the canonical data.

REAL-WORLD Like separating the 'order-taking' staff from the 'order-reading' staff so each scales independently.

Q39 What is Event Sourcing?

Instead of storing current state, store the full history of domain events. Replay events to reconstruct state. Clinical trials benefit from this — every AE status change is an event, providing a complete audit trail.

REAL-WORLD Like keeping a full CCTV recording of every change, so you can replay history, not just see 'now'.

Q40 What is the Backends for Frontends (BFF) pattern?

A dedicated gateway per client type (web BFF, mobile BFF). LifeTrack has a single Gateway.API. A BFF for the Angular app would aggregate and transform data specifically for the UI, reducing over-fetching.

REAL-WORLD Like a separate concierge desk for hotel guests vs convention attendees, each tailored to them.

Q41 How would you handle versioning of microservice APIs?

URL versioning (/api/v2/adverse-events), header versioning, or query string. When breaking changes are needed, run v1 and v2 concurrently. Gateway routes specific versions to specific service deployments.

REAL-WORLD Like product editions on a shelf (v1, v2) so existing customers' apps don't break overnight.

Q42 What is idempotency key? Why is it important in distributed systems?

A unique key in the request header that the server uses to detect duplicate requests. If the same RegisterPatient request is retried after a timeout, the idempotency key prevents creating two users. LifeTrack would add this to the registration saga.

REAL-WORLD Like a unique order number so re-submitting the same form doesn't double-charge you.

Q43 What is a Dead Letter Queue (DLQ)?

Messages that cannot be processed after N retries are moved to the DLQ for manual inspection. If LifeTrack used RabbitMQ for the patient-creation event, a DLQ would catch events that failed all retry attempts.

REAL-WORLD Like an 'undeliverable mail' tray for letters that failed after several delivery attempts.

Q44 What is the difference between API Gateway and Service Mesh?

API Gateway handles north-south traffic (external clients to services). Service mesh handles east-west traffic (service to service). LifeTrack's Ocelot is the API Gateway; a service mesh would handle internal calls between services.

REAL-WORLD Gateway is the building's front desk; service mesh is the internal phone system between offices.

Q45 What is polyglot persistence?

Using different database technologies per service based on its data access patterns. LifeTrack uses SQL Server for all services. Polyglot would allow Notification.API to use Redis (fast pub/sub), AnalyticsReport.API to use a time-series DB.

REAL-WORLD Like using a fridge for food and a safe for cash — the right storage for each kind of thing.

Q46 What is the shared database anti-pattern?

Multiple microservices sharing one database creates coupling — schema changes break other services. LifeTrack partially shares — governanceDb is shared by Auth and DocumentCompliance. The ideal is separate schemas or separate DB instances per service.

REAL-WORLD Like two unrelated shops sharing one till — a price change in one accidentally breaks the other.

Q47 What is service granularity?

How large or small a microservice should be. Too fine-grained: too many network calls. Too coarse-grained: loses benefits of microservices. LifeTrack has domain-aligned services (one per bounded context: adverse events, documents, analytics).

REAL-WORLD Like deciding how big each shop should be — too tiny means endless errands between them.

Q48 What is a bounded context in Domain-Driven Design?

A boundary within which a domain model is consistent. LifeTrack's AdverseEvent bounded context owns AdverseEventRecord and Deviation. Patient bounded context owns Patient and Enrollment. Each bounded context maps to a microservice.

REAL-WORLD Like one department that fully owns 'adverse events' — its own staff, forms and filing.

Q49 What is a saga vs a distributed transaction?

Distributed transaction uses 2PC locking (impractical across microservices). Saga breaks it into local transactions with compensating actions. LifeTrack's patient registration is a saga with 3 steps and rollback on failure.

REAL-WORLD A saga is a careful multi-step plan with undo buttons; a single DB transaction is one all-or-nothing switch.

Q50 What is the retry pattern? How would you implement it in LifeTrack?

Retry failed HTTP calls with exponential backoff. Use Polly library: services.AddHttpClient("PatientApi").AddTransientHttpErrorPolicy(p => p.WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)))).

REAL-WORLD Like politely redialing a busy number a few times, waiting longer each time.

Q51 What is the timeout pattern? Why is it critical in microservices?

Set a maximum wait time for inter-service calls. Without timeouts, a slow Patient.API causes AuthController to block indefinitely, exhausting thread pool. LifeTrack sets client.Timeout = TimeSpan.FromSeconds(10) on the PatientApi named client.

REAL-WORLD Like hanging up after 10 seconds on hold instead of waiting forever for the other shop.

Q52 What is a microservice chassis?

A reusable framework providing cross-cutting concerns: logging, tracing, health checks, config, JWT validation. Shared.CL is LifeTrack's chassis — services reference it and get JwtAuthFilter, GlobalExceptionMiddleware, etc. without re-implementing them.

REAL-WORLD Like the shared toolkit every shop bolts on — alarms, logs, ID-checks — built once, reused everywhere.

Q53 What monitoring tools would you use for LifeTrack in production?

Prometheus (metrics scraping from /metrics endpoints), Grafana (dashboards), Jaeger (distributed tracing), ELK stack (log aggregation). Health check endpoints (/health) feed into Kubernetes liveness/readiness probes.

REAL-WORLD Like a building dashboard showing each lift's status, temperature and alarm history.

Q54 What is the difference between liveness and readiness probes in Kubernetes?

Liveness: is the service alive? (restart if not). Readiness: is the service ready to serve traffic? (exclude from load balancer if not). LifeTrack services would use /health for readiness (includes DB connection check) and a simple ping for liveness.

REAL-WORLD Liveness = 'is the patient breathing?' (restart if not); readiness = 'ready to take visitors?'.

Q55 What is secret management in microservices?

Storing sensitive config (DB passwords, JWT secrets) securely. Options: Azure Key Vault, Kubernetes Secrets, HashiCorp Vault. LifeTrack reads JWT_SECRET from environment variables in production — better than hardcoding in appsettings.json.

REAL-WORLD Like a locked key-cabinet for the building's master keys instead of taping them under the desk.

Q56 What is the benefit of using separate DTOs per service?

Decouples the public API contract from the internal domain model. AdverseEvent.API's AdverseEventResponse DTO can evolve independently. Services are not coupled by shared model classes — a change in one DTO does not ripple across services.

REAL-WORLD Like each shop printing its own receipts so changing one shop's format never affects another.

Q57 What is a reverse proxy? How does Ocelot act as one?

A server that forwards client requests to backend servers and returns the response. Ocelot receives /gateway/adverse-events, rewrites the URL to /api/adverse-events, and proxies to localhost:5159. The client never directly reaches the backend service.

REAL-WORLD Like a hotel concierge forwarding your request to a back office and returning with the answer.

Q58 What is load balancing? How would it work with LifeTrack?

Distributing requests across multiple instances of a service. Ocelot supports load balancing (round robin, least connection) when multiple DownstreamHostAndPorts entries are configured. Production LifeTrack with 3 AdverseEvent.API instances would use this.

REAL-WORLD Like adding more checkout lanes and a queue manager during a sale rush.

Q59 What are the 12-Factor App principles most relevant to LifeTrack?

Config (store in env vars — JWT_SECRET), Codebase (one repo per service or monorepo), Dependencies (NuGet packages), Port binding (Kestrel on fixed ports), Stateless processes (no local state — JWT is stateless), Logs as event streams.

REAL-WORLD Like a tenant checklist: store keys in the safe, label rooms, keep your own utilities — portable, clean.

Q60 What is infrastructure as code (IaC)?

Managing infrastructure via code files (Terraform, Bicep). A Terraform module could provision the Azure SQL Server instances, App Service Plans, and AKS cluster for LifeTrack with reproducible, version-controlled infrastructure.

REAL-WORLD Like ordering a flat-pack building from a parts list so you can rebuild it identically anywhere.

Q61 How does LifeTrack achieve fault isolation between services?

Each service runs as a separate process. An unhandled exception in AdverseEvent.API (caught by GlobalExceptionMiddleware) does not crash Gateway.API or Authentication.API. Services are isolated at the OS process boundary.

REAL-WORLD Like each shop being a separate unit — a fire in one doesn't shut the whole mall.

Q62 What is the difference between vertical and horizontal microservice decomposition?

Horizontal: decompose by technical layers (frontend service, backend service, data service). Vertical: decompose by business capability (adverse events service, patient service). LifeTrack uses vertical decomposition — each service owns its full stack.

REAL-WORLD Horizontal = split by business (bakery, butcher); vertical = split by layer (front desk, storeroom).

Q63 What is Contract Testing in microservices?

Testing that service interactions conform to agreed API contracts without running all services together. Consumer-driven contract testing with Pact.io. LifeTrack would use this so Angular tests that /gateway/adverse-events returns the expected schema.

REAL-WORLD Like both companies signing off on the exact form layout before either changes it.

Q64 What is service-to-service authentication in microservices?

Services authenticating to each other (not just users). Options: mTLS, API keys, shared JWT secrets. LifeTrack's internal calls (Auth → Patient) use the user's JWT (forwarded with Bearer header). A service identity JWT (client_credentials flow) would be more secure.

REAL-WORLD Like staff badges proving one office is allowed to phone another (not just visitor wristbands).

Q65 What is the API composition pattern?

A query that joins data from multiple services. In LifeTrack, the Admin Dashboard shows protocols (ProtocolSite.API) + adverse events (AdverseEvent.API). The Angular frontend makes parallel API calls and composes the data client-side.

REAL-WORLD Like a concierge assembling your itinerary from the hotel, airline and car-hire desks into one sheet.

Q66 What is the scatter-gather pattern?

Send requests to multiple services in parallel, gather and aggregate responses. Angular's admin dashboard uses forkJoin (RxJS) to call multiple LifeTrack APIs in parallel — this is the client-side scatter-gather pattern.

REAL-WORLD Like sending runners to several shops at once and combining their replies on your desk.

Q67 What is a read model in CQRS? How would it apply to AnalyticsReport.API?

A denormalized view of data optimized for reads. AnalyticsReport.API's KpiReport is already a pre-aggregated read model — it stores computed metrics rather than querying raw data every time. This is CQRS read-side thinking.

REAL-WORLD Like a pre-printed summary report you read instantly, rather than recounting the whole ledger each time.

Q68 What is zero-trust architecture?

Never trust, always verify — authenticate every request even inside the network. LifeTrack's JwtAuthFilter on every service (not just the gateway) implements zero-trust — even internal calls must carry a valid JWT.

REAL-WORLD Like a building where every door checks ID even for staff — trust no one by default.

Q69 What is the retry storm problem?

If many services retry simultaneously after a failure, they amplify the load on the recovering service. Mitigated by adding jitter (random backoff). Polly's WaitAndRetryAsync with jitter would protect Patient.API from retry storms during LifeTrack registration.

REAL-WORLD Like everyone redialing a recovering hotline at once and crashing it again — add random pauses.

Q70 What is a token relay in microservices?

Forwarding the user's JWT from service to service so downstream services can authorize on behalf of the user. In LifeTrack, AuthController sets client.DefaultRequestHeaders.Authorization = "Bearer " + authResponse.Token before calling Patient.API.

REAL-WORLD Like the front desk forwarding your guest pass to the back office so it acts on your behalf.

Q71 What is the ambassador pattern?

A helper service or sidecar that handles network concerns (retry, circuit breaking, logging) on behalf of the main service. In Kubernetes with Istio, the Envoy sidecar is the ambassador. LifeTrack would benefit from this to remove retry logic from application code.

REAL-WORLD Like a personal assistant beside each manager who handles all their phone-calls and paperwork.

Q72 What is service decomposition by subdomain?

Using Domain-Driven Design subdomains to decide service boundaries. LifeTrack's core subdomain is clinical trial management; supporting subdomains are authentication (generic subdomain) and notifications (supporting subdomain).

REAL-WORLD Like organizing departments around real business areas (billing, shipping) not technical layers.

Q73 What is NoSQL vs SQL for microservices? When would you use each?

SQL (LifeTrack uses SQL Server): ACID, complex queries, structured data. NoSQL (MongoDB, Cosmos DB): flexible schema, horizontal scaling, document model. Notification.API could use Redis for fast pub/sub message storage.

REAL-WORLD SQL is a structured ledger with strict columns; NoSQL is a flexible scrapbook for varied notes.

Q74 What is the database migration challenge in microservices CI/CD?

Running migrations without downtime when multiple instances are running. Strategies: expand-contract pattern (add columns before removing old ones), blue-green with separate DB. LifeTrack uses dotnet ef database update which must run before new code is deployed.

REAL-WORLD Like renovating the plumbing without shutting off water — add new pipes before removing old ones.

Q75 What is consumer-driven contract testing?

The consumer (Angular) defines the contract it expects; the provider (AdverseEvent.API) verifies it meets the contract. Pact.io is the common tool. Catches breaking API changes before deployment.

REAL-WORLD Like the app team writing the exact API 'contract' the backend must satisfy before release.

Q76 What is event-driven architecture? How does Notification.API fit in?

Services communicate by publishing and subscribing to events. Notification.API (port 5210) receives notifications written to the shared DB by other services and serves them to the frontend. A full event-driven upgrade would use a message bus like Azure Service Bus.

REAL-WORLD Like a noticeboard: shops pin updates, others read them when they like (Notification.API serves the pins).

Q77 What is the Claim Check pattern?

Instead of embedding large payloads in messages, store them externally (blob storage) and pass a reference (claim check). Relevant if LifeTrack used messaging for document uploads — the message would carry the DocumentID, not the file bytes.

REAL-WORLD Like mailing a locker key instead of the heavy parcel — fetch the big item only if needed.

Q78 What is canary deployment?

Route a small percentage of traffic to the new version while the rest goes to the stable version. If metrics look good, gradually increase to 100%. Useful for deploying changes to LifeTrack's AdverseEvent.API without full rollout risk.

REAL-WORLD Like opening a new shop to a few customers first, watching, then welcoming everyone.

Q79 What is a sidecar pattern?

A co-deployed helper container alongside the main service container, handling cross-cutting concerns (logging, tracing, mTLS). In LifeTrack Kubernetes, Istio's Envoy proxy would be the sidecar for each service pod.

REAL-WORLD Like a personal assistant sitting beside each manager handling calls, security and notes.

Q80 What is the difference between a microservice and a nanoservice?

A nanoservice is too fine-grained — one function per service. This causes excessive network overhead and deployment complexity. LifeTrack avoids this by grouping related entities: AdverseEvent.API handles both AdverseEvents and Deviations (related concepts).

REAL-WORLD Like splitting a shop into 50 one-item kiosks — so many that running errands between them is absurd.

Q81 What are the LifeTrack microservice ownership boundaries?

Hari owns AdverseEvent.API (adverse events, deviations), DocumentCompliance.API (documents), AnalyticsReport.API (KPI reports). Naveen owns Patient.API (patients, enrollments, visits), ProtocolSite.API (protocols, sites, site-protocols).

REAL-WORLD Like a floor plan showing which team owns which department (Hari's wing vs Naveen's wing).

Q82 What is a correlation ID? How would you implement it in LifeTrack?

A unique ID stamped on a request at the gateway and forwarded downstream, linking all service logs for that request. Add CorrelationIdMiddleware in Gateway.API that generates X-Correlation-ID, passes it in downstream requests, and includes it in log output.

REAL-WORLD Like one tracking number stamped on a parcel so every depot's log can be stitched together.

Q83 What is blue-green deployment vs canary deployment vs rolling deployment?

Blue-green: two identical environments, instant switch. Canary: gradual traffic shift. Rolling: replace pods one by one. Blue-green gives instant rollback; canary minimizes blast radius; rolling uses least resources. LifeTrack in Kubernetes would use rolling for routine updates.

REAL-WORLD Blue-green = instant move to a twin house; canary = a few guests test first; rolling = redo rooms one by one.

Q84 What is the difference between a microservice and a service-oriented architecture (SOA)?

SOA services share an ESB (Enterprise Service Bus) and use heavyweight protocols (SOAP/WS-*). Microservices use lightweight HTTP/REST, each service is independently deployable, and there is no shared bus. LifeTrack is microservices; traditional hospital systems use SOA.

REAL-WORLD SOA shares one big switchboard (ESB); microservices each have their own phone — lighter and independent.

Q85 How would you implement centralized configuration for LifeTrack microservices?

Use Azure App Configuration or .NET's configuration providers with a central config server. All 8 LifeTrack services currently have their own appsettings.json — centralized config would remove duplication (JWT settings appear in all 8 services).

REAL-WORLD Like a single master settings binder all shops read from, instead of 8 separate copies.

Q86 What is the facade pattern in microservices?

The API Gateway acts as a facade — Angular talks to one URL (localhost:5046) and doesn't know about the 8 backend services. The facade simplifies the client interface and hides internal service topology.

REAL-WORLD Like a hotel lobby that hides the messy back offices behind one friendly front desk.

Q87 What is graceful degradation in microservices?

A service continues to function (with reduced capability) when a downstream dependency is unavailable. If Notification.API is down, LifeTrack should still allow clinical operations — notifications are non-critical. The system degrades gracefully by not failing the main operation.

REAL-WORLD Like a lift that still runs on reduced floors when one motor fails, rather than shutting completely.

Q88 What is the difference between failover and failback?

Failover: switch to a backup system when the primary fails. Failback: switch back to the primary after it recovers. In LifeTrack, if Patient.API fails during registration, the saga fails over to the compensating action (delete user). Failback happens when Patient.API restarts.

REAL-WORLD Failover = switch to the backup generator; failback = switch back to mains once power returns.

Q89 What is concurrency control in microservices?

Handling concurrent updates to shared data. Options: optimistic locking (add a RowVersion column, fail if version mismatch), pessimistic locking (DB row locks). LifeTrack document supersession logic (setting old docs to Superseded on new version upload) could benefit from optimistic locking.

REAL-WORLD Like two clerks editing one ledger — a version check stops one overwriting the other's entry.

Q90 What is mTLS (mutual TLS)?

Both client and server authenticate each other via TLS certificates. Used for service-to-service authentication in a zero-trust network. LifeTrack currently uses JWT for user auth but no service identity certificates — mTLS via Istio would be the production approach.

REAL-WORLD Like two parties each showing verified ID badges before talking — both sides prove who they are.

Q91 What are observability pillars in microservices?

Metrics (Prometheus — CPU, request rate, error rate), Logs (Serilog → Elasticsearch — structured log events), Traces (OpenTelemetry → Jaeger — distributed request trace). LifeTrack uses default ILogger currently; full observability would add all three.

REAL-WORLD Like a building's three monitors: power meters, the event logbook, and the parcel-tracking map.

Q92 What is a service registry?

A database of service instances and their locations. Consul, Eureka, or Kubernetes DNS. Services register on startup and deregister on shutdown. LifeTrack uses static URLs in config — a registry would enable dynamic service discovery for auto-scaling.

REAL-WORLD Like a live directory of which offices are open and where, updated as staff clock in and out.

Q93 How does LifeTrack's Notification.API work?

It serves notifications (port 5210) routed through the gateway at /gateway/notifications. Other services write notification records to the shared DB. The Angular frontend polls or subscribes to retrieve them. The DM badge count is persisted in the DB.

REAL-WORLD Like the mall noticeboard service that holds every shop's pinned messages and your unread count.

Q94 What is a microservice anti-pattern? Name examples from LifeTrack.

Shared database (Auth + DocumentCompliance share governanceDb — acceptable for LifeTrack scale but anti-pattern for large teams). Chatty services (too many fine-grained calls). Shared domain model across services. Log coupling.

REAL-WORLD Like two shops sharing one till (anti-pattern) — convenient now, painful when one needs changes.

Q95 How would you implement distributed caching across LifeTrack services?

Replace IMemoryCache with IDistributedCache backed by Redis (AddStackExchangeRedisCache). All service instances share the same cache — solves inconsistent in-memory state across multiple API instances. Redis also supports key pattern scanning, fixing the wildcard-removal issue.

REAL-WORLD Like a shared fridge (Redis) all branches use, so everyone sees the same stock instantly.

Q96 What is the GovernanceDb vs ClinicalDb split strategy in LifeTrack?

GovernanceDb (Authentication.API, DocumentCompliance.API): user management, auth, compliance documents — slower-changing, regulatory data. ClinicalDb (ProtocolSite.API, Patient.API, AdverseEvent.API, AnalyticsReport.API): operational clinical data with higher write volumes.

REAL-WORLD Like keeping HR files in the records room and live sales data in the shop floor system — separated by purpose.

Q97 What is a synchronous vs asynchronous saga? Which does LifeTrack use?

Synchronous saga: steps executed via direct HTTP calls in sequence (LifeTrack's RegisterPatient). Asynchronous saga: steps triggered by message bus events. Async is more resilient but harder to debug. LifeTrack uses synchronous for simplicity and explicitness.

REAL-WORLD Sync saga waits at each step (LifeTrack's way); async saga drops messages and reacts to events.

Q98 How does LifeTrack prevent duplicate user creation on RegisterPatient retry?

The User.Email column has a unique constraint in the DB. If a client retries after a timeout and Step 1 already succeeded, the second attempt returns a 409 Conflict (AuthException: email already registered). The partial create is cleaned up by the first attempt's compensation or is a no-op.

> **REAL-WORLD** Like a 'one membership per email' rule that blocks a duplicate sign-up if you submit twice.

Q99 What is horizontal pod autoscaling (HPA) in Kubernetes? How would LifeTrack use it?

HPA adds more pod replicas when CPU/memory thresholds are breached. LifeTrack would configure HPA for AdverseEvent.API with target CPU 70% — during a clinical trial's AE submission peak, Kubernetes adds replicas automatically, scaling back when load drops.

> **REAL-WORLD** Like opening extra checkout lanes automatically when the queue (CPU load) gets long.

Q100 What is the role of Shared.CL's RolesEnum vs each service's local UserRole enum?

Shared.CL.Enums.RolesEnum is used by cross-cutting auth filters (JwtAuthFilter, RoleAuthorizeFilter) shared across all services. Local UserRole enums mirror the DB role IDs for domain logic within each service. RolesEnum is the canonical source for authorization decisions.

> **REAL-WORLD** Like a master staff-roles list everyone uses, vs each shop's private cheat-sheet of the same roles.

Part III Angular (v20)

Components, forms, DI, routing, guards, HttpClient, RxJS and testing. · **110 questions**, each with an answer, project code where relevant, and a real-world analogy.

Q1 What is Angular? How is LifeTrack's Angular project structured?

Angular is a TypeScript SPA framework. Lifetrack.UI uses Angular (angular.json shows v19/20) with AppModule, feature modules per role (auth, dashboard), core (services, guards, interceptors, models), shared (reusable components), and environments.

REAL-WORLD Like a Lego city built from reusable bricks (components) snapped together by an instruction sheet (modules).

Q2 What is a Component in Angular? Give a LifeTrack example.

A component = template + TypeScript class + styles. LoginComponent (login.component.ts) has a form, loading state, and submits to AuthService. Decorated with @Component({selector, templateUrl, styleUrls}).

REAL-WORLD Like a single Lego brick: it has a shape (template), behaviour (class) and colour (styles).

Q3 What are the Angular lifecycle hooks? Which are most important?

ngOnChanges (input changes), ngOnInit (after first DI, most common), ngAfterViewInit (after view rendered), ngOnDestroy (cleanup). LifeTrack page components use ngOnInit to call load() methods (e.g., loadAdverseEvents()).

REAL-WORLD Like a person's day: wake up (OnInit), get a message (OnChanges), go to sleep (OnDestroy).

Q4 What is the difference between standalone components and module-based components?

Standalone components declare their own imports/providers without a module. Module-based components are declared in an NgModule. LifeTrack uses module-based (standalone: false in LoginComponent, DashboardModule, etc.) — traditional approach.

REAL-WORLD Standalone bricks come with their own studs; module bricks need a baseplate (NgModule) to attach.

Q5 What is data binding? Explain all four types with LifeTrack examples.

Interpolation {{user.name}} — display username. Property binding [disabled]="loading" — disables submit button. Event binding (click)="submit()" — form submission. Two-way binding [(ngModel)]="searchTerm" — search input updates and reads from component.

REAL-WORLD Like a smart price tag: shows the price ({{}}), greys out when sold ([disabled]), and updates as you type ([(ngModel)]).

Q6 What is Property Binding vs Attribute Binding?

Property binding [disabled]="loading" binds to the DOM property. Attribute binding [attr.aria-label]="label" binds to the HTML attribute when there is no DOM property equivalent. LifeTrack uses property binding extensively.

REAL-WORLD Property binding sets the price label; attribute binding sets the shelf's accessibility sign when there's no label slot.

Q7 **What is @Input() and @Output()? Give a LifeTrack example.**

@Input() allows a parent to pass data to a child component. @Output() emits events to the parent. PageHeaderComponent has @Input() title: string. EmptyStateComponent has @Input() message. Parent passes: <app-page-header [title]="'Adverse Events'">.

> **REAL-WORLD** Like a parcel passed down to a child (Input) and a receipt handed back up to the parent (Output).

Q8 **What is EventEmitter? How does it work with @Output()?**

EventEmitter is an Observable subclass used with @Output(). Child: @Output() saved = new EventEmitter<void>(); this.saved.emit(). Parent: <app-form (saved)="onSaved()">. Angular subscribes automatically.

> **REAL-WORLD** Like a doorbell the child presses (emit) that rings in the parent's house (subscribe).

Q9 **What is ViewChild? When would you use it in LifeTrack?**

@ViewChild('myInput') gives a reference to a child element or component. LifeTrack could use it to programmatically focus an input after modal opens: @ViewChild('titleInput') input!: ElementRef; ngAfterViewInit() { this.input.nativeElement.focus(); }

> **REAL-WORLD** Like grabbing a specific drawer handle by name to pull it open programmatically.

Q10 **What is the difference between ngOnInit and constructor in Angular?**

Constructor runs during class instantiation (before Angular processes @Input). ngOnInit runs after Angular has set inputs and performed DI. LifeTrack's LoginComponent builds the form in the constructor (safe, no async deps) and could load data in ngOnInit.

> **REAL-WORLD** Constructor is being born; ngOnInit is your first day at work once you've been given your tools.

Q11 **What are structural directives? Give LifeTrack examples.**

*ngIf="isLoading" — show spinner while loading. *ngFor="let ae of adverseEvents" — render AE table rows. *ngSwitch — conditionally render role-based content. They manipulate DOM structure by adding/removing elements.

> **REAL-WORLD** Like fold-out sections in a pop-up book that appear (*ngIf) or repeat per page (*ngFor).

Q12 **What is the difference between *ngIf and [hidden]?**

*ngIf removes/creates the DOM element. [hidden] keeps the element in DOM but sets display:none. *ngIf is better for expensive components (avoids creating them). [hidden] preserves state. LifeTrack uses *ngIf for conditional UI sections.

> **REAL-WORLD** *ngIf removes the chair from the room; [hidden] keeps the chair but throws a sheet over it.

Q13 **What is the async pipe? How would it work in LifeTrack?**

Auto-subscribes to an Observable and unsubscribes on component destroy. <div *ngIf="adverseEvents$ | async as events"> avoids manual subscribe/unsubscribe. LifeTrack currently uses subscribe() manually — async pipe would be cleaner.

> **REAL-WORLD** Like a magic mailbox that opens your letters automatically and shreds them when you leave.

Q14 What is ngClass? Give a LifeTrack example.

[ngClass]="{ 'badge-danger': ae.severity === 'Severe', 'badge-warning': ae.severity === 'Moderate' }" dynamically applies CSS classes to AE severity badges based on the value.

REAL-WORLD Like a mood ring that turns red for 'Severe' and amber for 'Moderate' based on the value.

Q15 What is ngStyle? How does it differ from ngClass?

ngStyle binds inline styles: [ngStyle]="{color: getStatusColor(doc.status)}". ngClass toggles CSS class names. ngStyle is better when values are dynamic (pixel values, colors). ngClass is better for predefined classes.

REAL-WORLD ngStyle paints a custom shade per item; ngClass picks from a set of pre-made paint pots.

Q16 What is a Custom Directive? Give a use case for LifeTrack.

A directive with custom logic applied via an attribute. @Directive({selector: "[ltHighlight]"}) that changes background color on mouse hover for table rows. Applied as <tr *ngFor="let ae of events" ltHighlight>.

REAL-WORLD Like a custom tooltip sticker you can slap onto any element to add hover behaviour.

Q17 What are built-in Angular pipes? Name five used or applicable to LifeTrack.

date ({{ae.reportedAt | date:"dd MMM yyyy"}}), uppercase ({{status | uppercase}}), currency ({{amount | currency:"INR"}}), number ({{count | number}}), percent ({{rate | percent}}). LifeTrack also uses a custom IstDatePipe for IST timezone conversion.

REAL-WORLD Like built-in phone filters: format a date, SHOUT text, add a ₹ sign.

Q18 What is a Custom Pipe? Describe LifeTrack's IstDatePipe.

@Pipe({name: "istDate"}) converts UTC DateTime strings to IST (UTC+5:30) for display. transform(value) adds 5.5 hours using IstHelper logic. Used in templates: {{event.reportedAt | istDate}}. Angular also configures DATE_PIPE_DEFAULT_OPTIONS with timezone "+0530".

REAL-WORLD Like a currency converter stamp that turns UTC times into Indian Standard Time on display.

Q19 What is the pure vs impure pipe distinction?

Pure pipes (default) run only when input reference changes. Impure pipes run on every change detection cycle. LifeTrack's IstDatePipe is pure — it transforms a string to a string, same input always gives same output.

REAL-WORLD Pure pipe = a vending machine (same coin, same snack); impure = a clock that re-checks every second.

Q20 What is the trackBy function in *ngFor? Why is it important?

Helps Angular identify which items changed, preventing full DOM re-render. trackBy: (index, ae) => ae.eventID. Without it, Angular destroys and recreates all DOM nodes on any array change. LifeTrack AE lists benefit from this for performance.

REAL-WORLD Like name tags on a relay team so the coach swaps only the runner who changed, not the whole team.

Q21 What is the difference between Template-Driven Forms and Reactive Forms?

Template-driven: form logic in HTML with ngModel (simpler, less testable). Reactive: form logic in TypeScript with FormBuilder (more testable, flexible, supports complex validation). LifeTrack's LoginComponent and RegisterComponent use Reactive Forms.

> **REAL-WORLD** Template-driven is a paper form you fill by hand; reactive is a smart digital form coded for testing.

Q22 What is FormBuilder? How did LifeTrack use it?

A service that creates FormGroup and FormControl instances. LoginComponent: this.form = this.fb.group({email: ['', [Validators.required, Validators.email]], password: ['', [Validators.required, Validators.minLength(6)]]}). Cleaner than manually newing up FormControls.

> **REAL-WORLD** Like a form-printing machine that stamps out validated fields from a blueprint.

Q23 What is FormGroup, FormControl, and FormArray?

FormGroup: groups multiple controls. FormControl: a single field. FormArray: a dynamic array of controls. LifeTrack uses FormGroup for the login/register forms. FormArray would be used for a form with dynamic protocol phase entries.

> **REAL-WORLD** FormControl = one field, FormGroup = the whole form, FormArray = a row you can keep adding.

Q24 What built-in validators does LifeTrack use?

Validators.required (all fields), Validators.email (email field), Validators.minLength(6) (password), Validators.maxLength(200) (name), Validators.maxLength(256) (email). Multiple validators are passed as an array.

> **REAL-WORLD** Like a form's red asterisks and 'enter a valid email' hints baked into each box.

Q25 What is a Cross-Field Validator? Give the LifeTrack example.

A validator on the FormGroup (not a single control) that can read multiple controls. LifeTrack's passwordsMatch validator: g.get("password").value !== g.get("confirm").value → return {mismatch: true}. Applied as: fb.group({...}, {validators: this.passwordsMatch}).

> **REAL-WORLD** Like a form that checks 'password' and 'confirm password' match before accepting.

Q26 What is a Custom Validator? Describe LifeTrack's dobValidator.

A function returning (control: AbstractControl) => ValidationErrors | null. dobValidator checks: DOB not in future, patient ≥ 18 years old, DOB not >120 years ago. Returns {dobFuture}, {dobMinAge}, or {dobMaxAge} accordingly.

> **REAL-WORLD** Like a custom age-check that rejects future birthdays and under-18s for a trial.

Q27 What is markAllAsTouched()? Why does LifeTrack use it?

Marks all form controls as "touched" so validation error messages appear. LifeTrack calls this.form.markAllAsTouched() before returning from submit() if the form is invalid — reveals all errors at once instead of only on blur.

> **REAL-WORLD** Like pressing 'submit' on an empty form and instantly seeing every box light up red.

Q28 What is the form.invalid guard in LifeTrack's submit()?

if (this.form.invalid) { this.form.markAllAsTouched(); return; } — prevents submitting an invalid form to the backend, marking all controls touched to display error messages.

REAL-WORLD Like a turnstile that won't let you through until the whole form is valid.

Q29 What is FormControl.touched vs dirty vs pristine?

Touched: user has blurred the field. Dirty: user has changed the value. Pristine: value unchanged from initial. LifeTrack shows validation errors only when a control is touched to avoid showing errors before the user interacts.

REAL-WORLD Touched = you clicked the box; dirty = you changed it; pristine = untouched since printing.

Q30 What is two-way binding with ngModel?

[(ngModel)] combines [ngModel] (property binding) and (ngModelChange) (event binding). Requires FormsModule. LifeTrack's search inputs use [(ngModel)]="searchTerm" for immediate two-way sync between the input and component property.

REAL-WORLD Like a live-mirroring text box — typing updates the variable and vice versa instantly.

Q31 What is Dependency Injection in Angular? How does providedIn: 'root' work?

Angular's DI container instantiates and injects services. providedIn: 'root' registers the service as a singleton at the root injector — shared across the app. AuthService is providedIn: 'root' so the same instance is shared by all components.

REAL-WORLD Like one shared office water-cooler (root) everyone uses instead of a cooler per desk.

Q32 What is hierarchical DI in Angular?

Services provided at different levels: root (app-wide singleton), module (shared within module), component (new instance per component). LifeTrack uses root-level for AuthService and other core services.

REAL-WORLD Like building managers at different levels: building-wide, floor-level, or per-room services.

Q33 What is AuthService in LifeTrack? What does it do?

Core service (providedIn: 'root'). Stores JWT in localStorage, exposes login()/logout()/isLoggedIn(), decodes token for role/userId, and exposes currentUser$ BehaviorSubject for reactive user state across the app.

REAL-WORLD Like the building's master keycard system — handles your login, ID and access everywhere.

Q34 What is a BehaviorSubject? How does LifeTrack use it?

An RxJS Subject that holds the current value and emits it to new subscribers. AuthService: currentUserSubject = new BehaviorSubject<UserInfo | null>(storedUser). Components subscribe to currentUser$ and get the current user immediately on subscribe.

REAL-WORLD Like a notice-board that always shows the latest memo and hands it to anyone who walks up.

Q35 What is the difference between Observable, Subject, and BehaviorSubject?

Observable: unicast, lazy (cold by default). Subject: multicast, no initial value. BehaviorSubject: multicast, holds current value, emits to new subscribers immediately. LifeTrack uses BehaviorSubject for currentUser$ because new components need the current state immediately.

> **REAL-WORLD** Observable = a radio you tune into; Subject = a megaphone; BehaviorSubject = a megaphone that repeats the last word to latecomers.

Q36 What is the role of environment.ts / environment.prod.ts?

Define environment-specific config. environment.ts: apiUrl: "http://localhost:5046/gateway". environment.prod.ts: apiUrl: "https://api.lifetrack.com/gateway". Selected at build time via angular.json fileReplacements. AuthService and other services reference environment.apiUrl.

> **REAL-WORLD** Like a settings switch: 'test mode' uses a local address, 'live mode' uses the real one.

Q37 What is Angular Router? How does LifeTrack configure it?

The Angular Router maps URL paths to components. AppRoutingModule defines top-level routes: /auth (lazy-loaded AuthModule), /dashboard (lazy-loaded DashboardModule, guarded by AuthGuard), ** redirects to /dashboard.

> **REAL-WORLD** Like a building's signpost system directing you to /auth or /dashboard.

Q38 What is lazy loading? How does LifeTrack use it?

Loading a module/component only when the route is first visited. loadChildren: () => import('./features/auth/auth.module').then(m => m.AuthModule). Reduces initial bundle size. LifeTrack lazy-loads both auth and dashboard feature modules.

> **REAL-WORLD** Like only printing a brochure section when a visitor first walks into that wing.

Q39 What is a Route Guard? What is CanActivate?

Guards control whether a route can be activated (CanActivate), deactivated (CanDeactivate), or loaded (CanLoad). AuthGuard implements CanActivate — if !auth.isLoggedIn(), it navigates to /auth/login and returns false.

> **REAL-WORLD** Like a security guard at a door deciding if you may enter (CanActivate).

Q40 What is AuthGuard in LifeTrack?

Protects the /dashboard route. Checks auth.isLoggedIn() (token exists and not expired). If not logged in, redirects to /auth/login. The DashboardModule has a child-level RoleGuard that further restricts routes by role.

> **REAL-WORLD** Like the guard who turns you away to the login desk if you have no valid pass.

Q41 What is RoleGuard? How does it work in LifeTrack?

A CanActivate guard that reads getRoleFromToken() from AuthService and checks if the user's role matches the required role for the route. If role mismatches, redirects to the role's default dashboard. Prevents Patients accessing CTM routes.

> **REAL-WORLD** Like a VIP-only guard who checks your pass says 'CTM' before letting you into the CTM lounge.

Q42 What is the difference between CanActivate and CanLoad?

CanActivate: checks before navigating to a route (module already downloaded). CanLoad: checks before downloading the lazy-loaded module — more efficient for unauthorized users. LifeTrack uses CanActivate; CanLoad would save bandwidth.

REAL-WORLD CanActivate checks at the door (already arrived); CanLoad checks before you even download the brochure.

Q43 What are Route Parameters vs Query Parameters?

Route params: /protocols/:id (part of URL path). Query params: /adverse-events?protocolId=5&page=2 (after ?). ActivatedRoute.snapshot.params["id"] for route params; ActivatedRoute.snapshot.queryParams["page"] for query params.

REAL-WORLD Route params = the house number in the address; query params = extra notes after a '?'.

Q44 What is the RouterModule.forRoot vs forChild?

forRoot registers the router with root-level providers (used once in AppRoutingModule). forChild registers routes without the root providers (used in feature modules like DashboardModule). LifeTrack uses forRoot in AppRoutingModule, forChild in DashboardRoutingModule.

REAL-WORLD forRoot is the building's main signpost (set once); forChild is a wing's local signpost.

Q45 What is programmatic navigation? How does LifeTrack use it?

router.navigate(["/dashboard"]) navigates without a link click. After login, LoginComponent calls this.router.navigate(["/dashboard"]). After logout, AuthService calls this.router.navigate(["/auth/login"]).

REAL-WORLD Like the app pressing the lift button for you instead of you tapping the screen.

Q46 What is a Resolver? When would LifeTrack use it?

A Resolver pre-fetches data before the route activates. LifeTrack could use a resolver for the Protocol Detail page to fetch the protocol before the component loads, ensuring the view renders with data immediately rather than showing a spinner.

REAL-WORLD Like a waiter bringing your starter to the table before you sit, so there's no wait.

Q47 What is Angular HttpClient? How does LifeTrack use it?

HttpClient makes HTTP requests and returns Observables. Provided via HttpClientModule in AppModule. AuthService: this.http.post<AuthResponse>(`${environment.apiUrl}/auth/login`, req). CtmAdverseEventsPage: this.http.get<PagedResult<AE>>(`${environment.apiUrl}/adverse-events`).

REAL-WORLD Like the building's postal service that sends and receives all letters (HTTP calls).

Q48 What is an HTTP Interceptor? What does AuthInterceptor do in LifeTrack?

An interceptor intercepts all HTTP requests/responses. AuthInterceptor clones the request and adds the Authorization: Bearer <token> header automatically, so every API call is authenticated without manually adding the header everywhere.

REAL-WORLD Like a mailroom clerk who stamps every outgoing letter with your address (token) automatically.

Q49 How is AuthInterceptor registered in LifeTrack?

In AppModule providers: { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }. multi: true allows multiple interceptors. The interceptor chain processes requests in order and responses in reverse order.

REAL-WORLD Like registering that clerk on the staff roster so every letter passes their desk.

Q50 What is the finalize() RxJS operator? How does LifeTrack use it?

Executes a callback whether the Observable completes or errors. LoginComponent: this.auth.login(req).pipe(finalize(() => this.loading = false)).subscribe(...). Ensures loading is set to false regardless of success/failure.

REAL-WORLD Like a 'please wait' spinner that always switches off whether the call succeeds or fails.

Q51 What is error handling in Angular HTTP? How does LifeTrack handle API errors?

In subscribe({ error: err => {...} }), LifeTrack extracts the error message: err.error?.error ?? err.error?.message ?? "default message". This handles the different error shapes from the backend (DomainException, AuthException, generic).

REAL-WORLD Like a help desk translating a cryptic error code into 'Invalid email or password'.

Q52 What is catchError in RxJS?

Catches an Observable error and returns a new Observable or throws. pipe(catchError(err => of(null))) returns null on error without propagating. LifeTrack could use catchError in services to handle errors before they reach the component.

REAL-WORLD Like a safety net that catches a dropped call and decides whether to retry or report it.

Q53 What is the difference between subscribe() and the async pipe for HTTP calls?

subscribe() is imperative — manually handle next/error/complete. Async pipe is declarative — template subscribes and auto-unsubscribes. LifeTrack uses subscribe(); async pipe is the modern best practice to prevent memory leaks.

REAL-WORLD subscribe is manually answering each call; async pipe is an answering machine that also hangs up for you.

Q54 What is switchMap? Give a LifeTrack use case.

Cancels the previous inner Observable when a new outer value arrives. If a user types in a search box and each keystroke triggers an API call, switchMap cancels the previous call. LifeTrack search components could use: searchTerm$.pipe(debounceTime(300), switchMap(term => this.api.search(term))).

REAL-WORLD Like a search box that cancels the half-typed query the moment you type a new letter.

Q55 What is forkJoin? How would LifeTrack use it?

Waits for all input Observables to complete, emits their last values as an array. Admin dashboard: forkJoin([this.getProtocols(), this.getAdverseEvents(), this.getUsers()]).subscribe(([protocols, events, users]) => {...}) — loads all data in parallel.

REAL-WORLD Like waiting for three separate deliveries to all arrive before you start cooking dinner.

Q56 What is retry() operator? How would it help LifeTrack?

Retries a failed Observable N times. this.http.get(...).pipe(retry(3)) retries up to 3 times on error. Useful for transient network failures when LifeTrack services are restarting.

REAL-WORLD Like auto-redialing a dropped call a few times before giving up.

Q57 What is state management in Angular? How does LifeTrack manage state?

Managing shared data across components. LifeTrack uses service-based state: AuthService holds currentUser$ (BehaviorSubject). DashboardService holds role-specific dashboard state. No NgRx currently — service state is sufficient for this scale.

REAL-WORLD Like a shared whiteboard the whole office reads and updates instead of passing notes.

Q58 What is NgRx? When would LifeTrack benefit from it?

Redux-inspired state management with Store, Actions, Reducers, Effects, Selectors. LifeTrack would benefit if the notification state or adverse event list state needs to be shared across multiple unrelated components without prop-drilling.

REAL-WORLD Like a city-wide filing system (NgRx) — overkill for a corner shop, vital for a chain.

Q59 What is the difference between NgRx and service-based state management?

NgRx: predictable, one-way data flow, devtools, time-travel debugging, but verbose. Services with BehaviorSubject: simpler, less code, sufficient for smaller apps. LifeTrack's scale suits service-based state; NgRx would be overkill.

REAL-WORLD NgRx is a strict bank ledger with audit trail; service-state is a personal notebook — pick by scale.

Q60 What are RxJS Observables? How are they different from Promises?

Observables are lazy, cancellable, and can emit multiple values. Promises eagerly execute and return one value. HttpClient.get() returns an Observable — LifeTrack can cancel a pending request by unsubscribing.

REAL-WORLD Observable is a Netflix stream you can pause/cancel; a Promise is a one-time pizza delivery.

Q61 What is the pipe() method in RxJS?

Chains operators applied to an Observable. loginObs.pipe(tap(res => storeToken(res)), finalize(() => loading = false)).subscribe(). Makes the transformation pipeline explicit and composable.

REAL-WORLD Like an assembly line of filters an item passes through before reaching you.

Q62 What is tap() in RxJS? How does AuthService use it?

Performs a side effect without transforming the value. AuthService: http.post().pipe(tap(res => { localStorage.setItem(TOKEN_KEY, res.token); currentUserSubject.next(res.user); })). Side effect: store token. Passes value through unchanged.

REAL-WORLD Like a side-task (save the receipt) done as the parcel passes, without opening the parcel.

Q63 What is debounceTime()? How would LifeTrack use it?

Waits for a specified quiet period before emitting. For a search input, debounceTime(300) waits 300ms after the user stops typing before calling the API — prevents a call per keystroke. LifeTrack search components should use this.

REAL-WORLD Like waiting 300ms after someone stops typing before searching, to avoid a search per keystroke.

Q64 What is distinctUntilChanged()? Give a LifeTrack use case.

Only emits when the value has actually changed. Combined with debounceTime: if the user types "adverse" then backspaces and retypes "adverse", distinctUntilChanged prevents a duplicate API call since the search term is the same.

REAL-WORLD Like ignoring a repeated search for the exact same word you just searched.

Q65 What is the difference between mergeMap, switchMap, concatMap, and exhaustMap?

mergeMap: runs all inner Observables concurrently. switchMap: cancels previous. concatMap: queues (sequential). exhaustMap: ignores new until current completes. LifeTrack form submit would use exhaustMap — ignore retaps while login is in progress.

REAL-WORLD Four ways to handle overlapping orders: all at once, cancel-previous, queue, or ignore-new-while-busy.

Q66 What is Subject in RxJS? How does it differ from Observable?

Subject is both Observable and Observer — you can subscribe to it and call .next() to emit. Observable is cold (creates new execution per subscriber). AuthService uses BehaviorSubject (a special Subject) to broadcast user state changes.

REAL-WORLD Observable is a radio station; Subject is a live mic anyone can both speak into and hear.

Q67 What is the takeUntil() operator? Why is it important?

Unsubscribes when a notifier Observable emits. Pattern: destroy$ = new Subject(); http.get().pipe(takeUntil(this.destroy$)).subscribe(); ngOnDestroy() { destroy$.next(); destroy$.complete(); } Prevents memory leaks. LifeTrack should use this.

REAL-WORLD Like an auto-unsubscribe that stops the radio when you leave the room, so it isn't left blaring.

Q68 What is a memory leak in Angular? How do you prevent it?

If a component subscribes to an Observable and doesn't unsubscribe on destroy, the subscription keeps the component alive in memory. Prevent with takeUntil(destroy$), async pipe, or using Subscription.unsubscribe() in ngOnDestroy(). LifeTrack should audit manual subscriptions.

REAL-WORLD Like leaving a tap running after you leave (leak) — takeUntil/async pipe turns it off on exit.

Q69 What is combineLatest()? Give a LifeTrack use case.

Emits the latest value of all inputs whenever any changes. For LifeTrack's filter: combineLatest([severity$, status$, page$]).subscribe(([severity, status, page]) => loadEvents(severity, status, page)) — auto-reloads when any filter changes.

REAL-WORLD Like a dashboard that refreshes the moment ANY of its three filters changes.

Q70 What is unit testing in Angular? What tools are used?

Testing individual components/services in isolation. Angular uses Jasmine (test framework), Karma (test runner), and TestBed (Angular testing module). LifeTrack would write spec files like login.component.spec.ts.

REAL-WORLD Like test-driving one car part on a rig (Jasmine/Karma) before fitting it.

Q71 What is TestBed in Angular testing?

TestBed.configureTestingModule creates a test Angular module. import components/services, provide mock dependencies, create component fixtures. LifeTrack: TestBed.configureTestingModule({declarations: [LoginComponent], providers: [{provide: AuthService, useValue: mockAuthService}]}).

REAL-WORLD Like a controlled test garage where you assemble just the part you want to test.

Q72 What is a ComponentFixture in Angular?

A wrapper around a component and its template. fixture.detectChanges() triggers change detection. fixture.nativeElement gives DOM access. fixture.componentInstance gives the component instance. Used in every Angular component unit test.

REAL-WORLD Like the test rig holding the component so you can poke it and watch it react.

Q73 How would you test LoginComponent?

Create fixture, inject mock AuthService that returns of({token: "test"}) from login(). Call component.form.patchValue({email:..., password:...}). Call component.submit(). Expect router.navigate called with ["/dashboard"].

REAL-WORLD Like rehearsing 'enter email, click submit, expect redirect' with a stand-in login service.

Q74 What is E2E testing in Angular? What tool is commonly used?

End-to-end tests simulate real user interactions in a browser. Cypress or Playwright are modern choices (Protractor deprecated). LifeTrack E2E: navigate to /auth/login, fill form, click submit, assert /dashboard URL.

REAL-WORLD Like a full dress rehearsal walking through the whole app as a real user would (Cypress).

Q75 What is Jasmine? What is the structure of a Jasmine test?

Jasmine is a BDD test framework. describe("LoginComponent", () => { it("should call auth.login on submit", () => { expect(authSpy.login).toHaveBeenCalledWith({email:..., password:...}); }); })

REAL-WORLD Like a checklist script: 'describe the login, it should redirect, expect this to happen'.

Q76 What is a spy in Jasmine?

A test double that tracks calls. spyOn(authService, "login").and.returnValue(of(mockResponse)). Lets you assert the method was called and control its return value. LifeTrack tests use spies on AuthService.login and Router.navigate.

REAL-WORLD Like a stunt double that stands in for the real service and records every time it's called.

Q77 How would you test AuthService in LifeTrack?

Use HttpClientTestingModule + HttpTestingController. Call authService.login(req). Expect a POST request to /auth/login. Flush mock response. Assert localStorage has the token. Assert currentUser$ emits the user.

REAL-WORLD Like rehearsing the login service with a fake postbox (HttpTestingController) and checking the stored key.

Q78 What is the Angular CLI? What commands are most used in LifeTrack?

ng serve (dev server), ng build --prod (production build), ng generate component (create component), ng test (run unit tests), ng lint (code quality). LifeTrack uses ng serve during development and ng build --configuration production for deployment.

REAL-WORLD Like the workshop tools: ng serve (test drive), ng build (ship), ng generate (stamp a new part).

Q79 What is Change Detection in Angular? What are the two strategies?

Angular re-renders when data changes. Default: checks all components on any change. OnPush: only checks when @Input reference changes, async pipe emits, or events. LifeTrack uses Default. OnPush would improve performance for large tables.

REAL-WORLD Default re-checks the whole house on any noise; OnPush only checks rooms whose key (input) changed.

Q80 What is an Angular Module (NgModule)? What does AppModule declare?

NgModule groups declarations (components), imports (other modules), providers (services), and bootstrap (root component). AppModule declares AppComponent, imports BrowserModule + HttpClientModule + AppRoutingModule, provides HTTP_INTERCEPTORS (AuthInterceptor).

REAL-WORLD Like the building's master registry listing every room, shared facility and service.

Q81 What is the difference between declarations, imports, exports, and providers in NgModule?

Declarations: components/directives/pipes belonging to this module. Imports: other modules needed. Exports: make declarations available to other modules. Providers: register services for this module's injector.

REAL-WORLD declarations = rooms you own, imports = shared facilities you use, exports = rooms you lend, providers = your services.

Q82 What is SharedModule in LifeTrack?

Contains reusable components (PageHeaderComponent, EmptyStateComponent) and re-exports common modules (CommonModule, FormsModule, ReactiveFormsModule). Imported by every feature module to avoid repetitive imports.

REAL-WORLD Like a shared supplies cupboard (headers, empty-state) every department borrows from.

Q83 What is APP_INITIALIZER?

A token that runs a function before the app starts. LifeTrack could use it to pre-load the user profile from the stored JWT before any component renders, ensuring the currentUser$ has a value on first paint.

REAL-WORLD Like the building doing a safety check before opening its doors each morning.

Q84 What is the DATE_PIPE_DEFAULT_OPTIONS token? How does LifeTrack use it?

Configures default options for DatePipe globally. LifeTrack provides: { provide: DATE_PIPE_DEFAULT_OPTIONS, useValue: { timezone: '+0530' } } in AppModule — all DatePipe instances format dates in IST by default.

> **REAL-WORLD** Like a building-wide rule that all clocks display Indian Standard Time.

Q85 What is the role of the AuthInterceptor in LifeTrack?

It intercepts every outgoing HTTP request and adds the Authorization: Bearer <token> header. This means all services (AdverseEventService, DocumentService, etc.) can make unauthenticated HTTP calls — the interceptor transparently authenticates them.

> **REAL-WORLD** Like the mailroom clerk who stamps every letter — the reason no department forgets the address.

Q86 What is JWT decoding on the Angular client? How does AuthService do it?

AuthService.decodeToken() splits the JWT on ".", base64-decodes the payload (padding-safe atob), and JSON.parses it. getRoleFromToken() reads the role claim. This is client-side only — the backend re-validates the signature on every request.

> **REAL-WORLD** Like reading the verified info printed on your festival wristband instead of trusting your own scribble.

Q87 Why is reading role from localStorage less secure than reading from the JWT?

localStorage values can be edited by users in browser DevTools. The JWT payload is encoded (not encrypted) but the signature verification happens server-side. AuthService reads role from the JWT payload, not from localStorage user object — comment explains this security reason.

> **REAL-WORLD** Trusting your handwritten badge (localStorage) is risky; the holographic wristband (JWT) is verified by security.

Q88 What is the isLoggedIn() method in LifeTrack's AuthService?

Checks: (1) token exists in localStorage, (2) token expiry (exp claim × 1000) is in the future. If exp is null (no exp claim), returns true conservatively. AuthGuard calls this on every dashboard route access.

> **REAL-WORLD** Like a turnstile that checks both 'do you have a pass' and 'has it expired'.

Q89 What is the role of finalize() vs complete() in LifeTrack forms?

finalize() runs after the Observable completes or errors — always executed. complete() is a notification from the source. LifeTrack uses finalize(() => this.loading = false) to guarantee the spinner stops regardless of API success or failure.

> **REAL-WORLD** finalize is 'switch off the spinner no matter what'; complete is 'the call finished successfully'.

Q90 What is the handleAuthResponse method in AuthService?

Stores the token and user from an AuthResponse (without calling login() again). Used after patient self-registration — the backend returns a JWT in the 201 response, and handleAuthResponse stores it so the patient is immediately logged in.

> **REAL-WORLD** Like a fast-track: after sign-up the desk hands you your pass directly, no second queue.

Q91 What is the role-based dashboard routing in LifeTrack?

After login, the DashboardComponent (or a router guard) reads the user's role and redirects to the role-specific component: Admin → AdminDashboard, CTM → CtmDashboard, Investigator → InvestigatorDashboard, etc. Each role sees a tailored UI.

REAL-WORLD Like a hotel directing each guest type (VIP, staff, visitor) to their own floor after check-in.

Q92 What are the roles in LifeTrack and what can each access?

Admin (full access), ClinicalTrialManager (protocols, sites, AEs, deviations, documents, reports), Investigator (patients, visits, enrollments, documents), RegulatoryOfficer (documents, compliance dashboard, KPI reports), DataManager (AEs, deviations, reports), Patient (own enrollment, visits, portal).

REAL-WORLD Like six staff badges each opening a different set of doors (Admin, CTM, Patient...).

Q93 What is Angular CLI's ng build --configuration production? What does it do?

Performs ahead-of-time (AOT) compilation, tree-shaking (removes unused code), minification, file hashing for cache-busting, and uses environment.prod.ts. Generates optimized bundles in the dist/ folder for deployment.

REAL-WORLD Like a factory's final polish-and-shrink step before the product ships to stores.

Q94 What is AOT (Ahead-of-Time) compilation in Angular?

Compiles Angular templates to JavaScript at build time (not at runtime in the browser). Benefits: faster startup, smaller bundle (no Angular compiler in bundle), earlier template error detection. ng build --prod uses AOT by default.

REAL-WORLD Like pre-cooking meals in the factory (build time) instead of cooking in the customer's kitchen.

Q95 What is tree-shaking?

Removing unused code from the bundle during build. If a module is imported but never used, tree-shaking eliminates it. LifeTrack's production build uses tree-shaking to reduce bundle size.

REAL-WORLD Like packing only the clothes you'll wear, leaving unused items out of the suitcase.

Q96 What is the angular.json file?

Angular workspace configuration. Defines projects, build targets, file replacements (environment.ts → environment.prod.ts for prod), style preprocessors, and test configuration. LifeTrack's angular.json configures the Lifetrack.UI project.

REAL-WORLD Like the master blueprint file telling builders how to assemble and ship the app.

Q97 What is the difference between ngModel and FormControl for validation?

ngModel (template-driven): validation via HTML attributes (required, minlength). FormControl (reactive): validation via Validators array in TypeScript. LifeTrack uses FormControl for testability — validators are plain functions, easy to unit test.

REAL-WORLD Hand-written paper form (ngModel) vs a coded digital form (FormControl) that's easy to test.

Q98 What is abstractControl.hasError() vs abstractControl.getError()?

hasError("required") returns a boolean. getError("required") returns the error value (e.g., {requiredLength: 6, actualLength: 3} for minlength). LifeTrack templates use hasError() for *ngIf on error messages.

REAL-WORLD hasError just asks 'is it wrong?'; getError asks 'how exactly is it wrong?' (e.g. min length 6).

Q99 What is the FormGroup-level error vs FormControl-level error?

Control-level: Validators on individual controls (required, email). Group-level: validators on the FormGroup comparing multiple controls. LifeTrack's passwordsMatch is group-level: this.form.hasError("mismatch") not this.confirm.hasError("mismatch").

REAL-WORLD Control-level = one field's error; group-level = a 'passwords don't match' error across two fields.

Q100 What is HttpParams in Angular?

A utility for building URL query strings in a fluent, immutable way. new HttpParams().set("page", "1").set("severity", "Mild"). LifeTrack services use query params directly in URL template literals or could use HttpParams for cleaner code.

REAL-WORLD Like a tidy URL-builder that safely appends '?page=1&severity=Mild' to a web address.

Q101 What is providedIn: 'root' vs providing in a module?

providedIn: 'root': single shared instance (singleton) for the entire app. Providing in a module: a new instance per module (useful for feature-scoped state). AuthService is root-singleton. If DashboardService needed CTM-scoped state, it would be provided in DashboardModule.

REAL-WORLD Root = one shared cooler for all; module-provided = a fresh cooler per department.

Q102 What is the difference between HttpClientModule and provideHttpClient()?

HttpClientModule is the NgModule-based approach (used in AppModule imports). provideHttpClient() is the functional API for standalone applications. LifeTrack uses HttpClientModule since it is module-based.

REAL-WORLD HttpClientModule is the classic baseplate; provideHttpClient() is the new snap-on for standalone apps.

Q103 What is an Observable cold vs hot?

Cold: execution starts fresh per subscriber (HttpClient.get() — each subscription makes a new HTTP request). Hot: shared execution (Subject — all subscribers see the same values). AuthService.currentUser$ (BehaviorSubject) is hot — one underlying stream, multiple subscribers.

REAL-WORLD Cold = a fresh Netflix stream per viewer; hot = a live TV broadcast everyone shares.

Q104 What is ActivatedRoute? How would LifeTrack use it?

Provides access to route params, query params, and data for the active route. A Protocol Detail component would inject ActivatedRoute and use this.route.snapshot.params["id"] or this.route.paramMap.pipe(switchMap(...)) to load the correct protocol.

REAL-WORLD Like a desk clerk reading which file (route) you asked for to fetch the right record.

Q105 What is the role of Validators.compose()?

Combines multiple validators into one. Validators.compose([Validators.required, Validators.email]) is equivalent to passing them as an array in FormBuilder. LifeTrack uses the array shorthand directly.

> **REAL-WORLD** Like stapling several validation rules together into one rubber stamp.

Q106 What is Angular's zone.js and how does it relate to change detection?

zone.js patches async APIs (setTimeout, XHR, Promises) and notifies Angular when async operations complete, triggering change detection. This is why Angular re-renders automatically after an HTTP call completes without manual markForCheck().

> **REAL-WORLD** Like the building's nervous system that senses any movement and tells the manager to re-check.

Q107 What is the async/await pattern vs Observable pattern in Angular services?

Async/await uses Promises (native JS). Observables (RxJS) offer more operators, cancellation, and composability. LifeTrack services return Observables (HttpClient returns Observable). Using firstValueFrom() converts to a Promise if needed.

> **REAL-WORLD** async/await is a one-time pizza order; Observables are a subscription with cancel and replay options.

Q108 What is ngOnDestroy? Why is it important in LifeTrack components?

Lifecycle hook called when a component is destroyed. Use to unsubscribe from Observables, clear timers, and release resources. LifeTrack dashboard components that use subscribe() should implement ngOnDestroy to prevent memory leaks when navigating between sections.

> **REAL-WORLD** Like turning off the lights and tap when you leave a room, so nothing's left running.

Q109 What is the purpose of the empty-state component in LifeTrack?

A reusable shared component that displays a friendly message when a list has no items. Takes @Input() message: string. Used across all list pages: <app-empty-state [message]="'No adverse events found'"> prevents blank pages.

> **REAL-WORLD** Like a friendly 'No records found' sign instead of an awkward blank wall.

Q110 What is page-header component in LifeTrack?

A shared reusable component that renders a consistent page title and optional description. Takes @Input() title and @Input() subtitle. Ensures all dashboard pages have a uniform heading style without duplicating HTML.

> **REAL-WORLD** Like a consistent page banner at the top of every department's notice page.